

09:00-09:50 (쉬는 시간·점심시간 11:30-13:00)

회의 Agent 전체 지도

LLM · Workflow · 01_architecture.json

진행

회의 Agent 전체 지도
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/
day2_meeting_workflow.py
materials/day2/ 일반인을_위한_회의기록
_Agent_설계.md

확인할 결과

01_architecture.json

Well-being Meeting Record 완성 화면

요약 한 번을 Agent라 부르면 비용·오류·승인 위치를 설명할 수 없다.

입력 계약

→ 핵심 처리

→ 실패 경계

→ 사람 판단

→ 평가·실행 증거

06 검토할 결과

원문과 담당자·기한을 확인한 뒤에만 저장하거나 보내세요.

검토 준비

회의 기록 초안이 준비되었습니다

Google Meet 또는 전사 TXT·인타넷 없이 연승을 결과

문서 저장

전체 결과 저장

아직 저장하거나 보내지 않았습니다. 원문, 담당자, 기한을 사람이 확인해야 합니다.

상황 판단 · Agent

이번 요청에 사용한 방식

외부 정보·도구·후속 전달 가능성이 있는 추가 요청이라 계획과 승인을 먼저 분리했습니다.

요청 목적 판단

필요한 정보와 연결 후보 계획

허용된 회의 입력만 사용

선택 결과 생성

외부 작업 계획 분리

근거 번호 검사

사람 검토 대기

8 원문 구간

3 확인할 참석자

5 할 일

7 연결된 근거

회의 기록

문서 미리보기

이메일 초안

연결 계획

원문 근거

회의 기록과 후속 실행

회의에서는 다음 내용을 중심으로 논의했습니다. 오늘은 고객 상담 요약 기능의 1차 출시 범위와 검증 기준을 정하겠습니다. 상담 후 요약, 고객 요청, 상담사가 할 일을 한 화면에 보여주는 범위가 적절합니다. 개인정보가 들어갈 수 있으니 원문 보관 기간과 마스킹 기준을 먼저 확인해야 합니다. 1차 출시는 자동 발송 없이 상담사가 결과를 확인하고 저장하는 방

재생 파일 · assets/demo-videos/day2_service_teaser.gif

오늘 8차시는 이 화면을 현재 저장소에서 다시 만드는 과정입니다.

2일차 전체 시간표

1~3차시 · 쉬는 시간과 점심시간 · 4~5차시 · 쉬는 시간 20분 · 6~8차시 · Q&A 30분

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	회의 Agent 전체 지도	강의 12 시연 10 Codex 제작 23 확인 5분	01_architecture.json
09:50-10:40	2차시	세 가지 입력과 STT 선택	강의 12 시연 10 Codex 제작 23 확인 5분	02_inputs.json
10:40-11:30	3차시	도메인 맥락과 MCP 정책	강의 12 시연 10 Codex 제작 23 확인 5분	03_domain_context.json
13:00-13:50	4차시	MeetingRecord 결과 계약	강의 12 시연 10 Codex 제작 23 확인 5분	04_strategy_compare.json
13:50-14:40	5차시	Codex·Claude 시나리오 설계	강의 12 시연 10 Codex 제작 23 확인 5분	05_workflow_runs.json
15:00-15:50	6차시	LLM Provider와 비용 경계	강의 12 시연 10 Codex 제작 23 확인 5분	06_provider_diagnostics.json
15:50-16:40	7차시	LangGraph 사람 검증	강의 12 시연 10 Codex 제작 23 확인 5분	07_human_review.json
16:40-17:30	8차시	Desktop 선물 패키지	강의 12 시연 10 Codex 제작 23 확인 5분	08_export_drafts.json

11:30-13:00 쉬는 시간·점심시간 · 14:40-15:00 쉬는 시간 · 17:30-18:00 쉬는 시간·Q&A

회의 Agent 전체 지도 학습 결과

- 01 — LLM 호출·고정 Workflow·Agent 판단을 구분하고 상황에 맞는 제품 경계를 그린다.
- 02 — 모든 단계를 상위 모델에게 맡겨 같은 입력에도 비용과 결과가 달라진다. 왜 위험한지 설명한다.
- 03 — 01_architecture.json에서 정상과 실패 상태를 직접 확인한다.

회의 Agent 전체 지도 용어

용어	이 수업에서 쓰는 뜻
LLM	주어진 맥락으로 하나의 결과를 만드는 모델
Workflow	순서와 실패 규칙을 코드로 고정한 흐름
Agent	상황을 보고 다음 도구와 순서를 선택하는 시스템
Human gate	외부 쓰기 전 사람이 확정하는 경계

01_architecture.json 입출력

INPUT

src/course_services/day2_meeting_workflow.py

OUTPUT

01_architecture.json

PROCESS

사용자 요청 → 입력 유형 판단 → 고정 처리

VERIFY

요청이 LLM·Workflow·Agent 중 어디에 속하는지
이유와 함께 반환된다.
자동 발송·게시 요청은 external_write=false와
APPROVAL_REQUIRED로 끝난다.

회의 Agent 전체 지도 실행 흐름

01

사용자 요청

02

입력 유형 판단

03

고정 처리

04

필요한 사고만 LLM

05

사람 확정

마지막 판단은 01_architecture.json에서 사람이 확인합니다.

01_architecture.json 정상·실패 계약

정상	—— 요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다.
경계	—— 자동 발송·게시 요청은 external_write=false와 APPROVAL_REQUIRED로 끝난다.
외부 쓰기	—— 회의 Agent 전체 지도: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·01_architecture.json

회의 Agent 전체 지도 파일 지도

입력·fixture

src/course_services/day2_meeting_workflow.py

구현 코드

materials/day2/일반인을_위한_회의기록_Agent_설계.md

검증·test

tests/test_day2_meeting_workflow.py

따라하기 notebook

materials/day2/day2_service_lab.ipynb

회의 Agent 전체 지도 개념·코드

LLM

주어진 맥락으로 하나의 결과를 만드는 모델

```
src/course_services/day2_meeting_workflow.py
```

Agent

상황을 보고 다음 도구와 순서를 선택하는 시스템

```
tests/test_day2_meeting_workflow.py
```

Workflow

순서와 실패 규칙을 코드로 고정한 흐름

```
materials/day2/일반인을_위한_회의기록_Agent_설계.md
```

Human gate

외부 쓰기 전 사람이 확정하는 경계

```
materials/day2/day2_service_lab.ipynb
```

회의 Agent 전체 지도 선택 기준

구분	역할	이번 차시의 판단 기준
LLM	사용자 요청 단계의 입력 기준	결정론 test로 먼저 확인
Workflow	입력 유형 판단 단계의 교체 경계	adapter 경계를 확인
Agent	고정 처리 단계의 운영 조건	비용·지연·권한을 확인
Human gate	필요한 사고만 LLM 단계의 판단 기록	사람 판단과 운영 기록을 확인

회의 Agent 전체 지도 대표 실패

실패

모든 단계를 상위 모델에게 맡겨 같은 입력에도 비용과 결과가 달라진다.

사용자 영향

모든 단계를 상위 모델에게 맡겨 같은 입력에도 비용과 결과가 달라진다.
상태에서는 01_architecture.json을 운영 판단에 사용할 수 없다.

회의 Agent 전체 지도 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

회의 Agent 전체 지도 복구 경로

- 01 — 탐지: 자동 발송·게시 요청은 `external_write=false`와 `APPROVAL_REQUIRED`로 끝난다.
- 02 — 중단: 모든 단계를 상위 모델에게 맡겨 같은 입력에도 비용과 결과가 달라진다.
- 03 — 복구: 입력 검사·STT·schema·증거 검증은 코드로 고정하고 요약·맥락 해석만 모델에게 맡긴다.
- 04 — 증거: `01_architecture.json`과 test 결과를 함께 남긴다.

회의 Agent 전체 지도 Codex 검증 루프

materials/day2/일반인을_위한_
회의기록_Agent_설계.md의
책임을 찾고
01_architecture.json 계약을
focused test로 고정합니다.

회의 Agent 전체 지도
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



회의 Agent 전체 지도 Codex 작업 명세

목표

세 입력 시나리오와 외부 쓰기 금지를 명세하고 가장 작은 실행 구조를 제안받는다.

허용 경로

materials/day2/ 일반인을_위한_회의기록_Agent_설계.md
tests/test_day2_meeting_workflow.py

완료 조건

요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다.
자동 발송·게시 요청은 external_write=false와 APPROVAL_REQUIRED로 끝난다.

금지

회의 Agent 전체 지도: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

회의 Agent 전체 지도 독립 리뷰

- | | | | |
|----|---------------|----|---|
| 01 | Codex diff | —— | 회의 Agent 전체 지도 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다. |
| 03 | Claude review | —— | 01_architecture.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 회의 Agent 전체 지도의 두 LLM 의견과 test는 자동 승인이 아니다 |

회의 Agent 전체 지도 정상 시연

- 01 — 열기: `src/course_services/day2_meeting_workflow.py`
- 02 — 구현 확인: `materials/day2/일반인을_위한_회의기록_Agent_설계.md`
- 03 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py`
- 04 — 성공 신호: 요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다.

회의 Agent 전체 지도 실행 명령

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py
```

실행 전 확인

회의 Agent 전체 지도: repository root · Python 3.12 · .env 미출력

01_architecture.json 실행 결과

```
{
  "user_request": "회의를 이해하고 근거 있는 기록·할 일·인사이트 초안을 만들어 줘",
  "layers": [
    {
      "order": 1,
      "name": "policy",
      "question": "읽어도 되는 정보와 하면 안 되는 행동은?"
    },
    {
      "order": 2,
      "name": "input_adapter",
      "question": "Meet·ClovaNote·음성 중 어떤 한 입력인가?"
    },
    {
      "order": 3,
      "name": "domain_context",
      "question": "산업 용어와 이전 결정은 무엇인가?"
    },
    {
      "order": 4,
      "name": "workflow",
      "question": "정규화→STT→구조화→근거 검증 순서는?"
    },
    {
      "order": 5,
      ...
    }
  ]
}
```

회의 Agent 전체 지도 실패·복구 시연

재현

자동 발송·게시 요청은
external_write=false와
APPROVAL_REQUIRED로 끝난다.

복구

입력 검사·STT·schema·증거 검증은
코드로 고정하고 요약·맥락 해석만
모델에게 맡긴다.

확인: 자동 발송·게시 요청은 external_write=false와 APPROVAL_REQUIRED로 끝난다.

회의 Agent 전체 지도 소프트웨어 제작

열 파일

materials/day2/일반인을_위한_회의기록_Agent_설계.md
tests/test_day2_meeting_workflow.py

작업 범위

회의 Agent 전체 지도 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다.
01_architecture.json 저장

회의 Agent 전체 지도 Notebook 셀

materials/day2/day2_service_lab.ipynb · 1차시

```
from src.course_services.day2_meeting_workflow import (
    DEFAULT_OPENAI_MODEL, DomainContext, MCPRetrievalPolicy, MeetingRecord,
    SourceInput, TranscriptEnvelope, build_mcp_retrieval_plan,
    compact_workflow_result, compare_execution_strategies,
    diagnose_provider_options, normalize_source, render_email_draft,
    route_execution_strategy, run_meeting_workflow,
    run_optional_cli_prompt, run_optional_openai_prompt, run_optional_openai_record,
    source_mixing_error_example,
    validate_record_evidence,
)

architecture = {
    "user_request": "회의를 이해하고 근거 있는 기록·할 일·인사이트 초안을 만들어 줘",
    "layers": [
        {"order": 1, "name": "policy", "question": "읽어도 되는 정보와 하면 안 되는 행동은?"},
        {"order": 2, "name": "input_adapter", "question": "Meet·ClovaNote·음성 중 어떤 한 입력인가?"},
        {"order": 3, "name": "domain_context", "question": "산업 용어와 이전 결정은 무엇인가?"},
        {"order": 4, "name": "workflow", "question": "정규화→STT→구조화→근거 검증 순서는?"},
        {"order": 5, "name": "human_review", "question": "누가 승인·수정·거절하는가?"},
        {"order": 6, "name": "draft_export", "question": "MD·이메일 초안을 어디까지 만들 것인가?"},
    ]
}

# ... 다음 줄은 Notebook에서 계속
```

회의 Agent 전체 지도 Terminal 실행

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py
```

저장 위치

```
output/course-labs/day2-v2/01_architecture.json
```

요청이 LLM-Workflow-Agent 중 어디에 속하는지 이유와 함께 반환된다.
실패 시 01_architecture.json의 status·error_code를 먼저 확인

회의 Agent 전체 지도 실패 증거

자동 발송·게시 요청은 `external_write=false`와 `APPROVAL_REQUIRED`로 끝난다.

```
result_file = "output/course-labs/day2-v2/01_architecture.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

01_architecture.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

회의 Agent 전체 지도 정상 Case

NORMAL

요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다.

- 01 — src/course_services/day2_meeting_workflow.py 입력과 command가 기록됐는가
- 02 — 01_architecture.json schema가 validate되는가
- 03 — 회의 Agent 전체 지도 focused test가 실제로 통과했는가

회의 Agent 전체 지도 경계 Case

BOUNDARY

자동 발송·게시 요청은 `external_write=false`와 `APPROVAL_REQUIRED`로 끝난다.

- 01 — 회의 Agent 전체 지도: raw traceback 대신 stable error_code가 남는가
- 02 — 01_architecture.json: 외부 쓰기·자동 메일이 false인가
- 03 — 자동 발송·게시 요청은 `external_write=false`와 `APPROVAL_REQUIRED`로 끝난다. 재현이 가능한가

회의 Agent 전체 지도 현업 적용

비개발자도 한눈에 읽는 회의기록 제품 설계도

- 01 — 회의 Agent 전체 지도은 합성·비식별 sample로 먼저 실행
- 02 — 비개발자도 한눈에 읽는 회의기록 제품 설계도의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 01_architecture.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 01_architecture.json을 운영 검토 자료로 사용

회의 Agent 전체 지도 포트폴리오 적용

회의 Agent 전체 지도 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 모든 단계를 상위 모델에게 맡겨 같은 입력에도 비용과 결과가 달라진다.의 사용자 영향을 한 문장으로 설명
- 02 — 회의 Agent 전체 지도 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 세 입력 시나리오와 외부 쓰기 금지를 명세하고 가장 작은 실행 구조를 제안받는다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 01_architecture.json과 README 재현 절차를 제출

회의 Agent 전체 지도 운영 점검

품질

무엇을 요청이 LLM·Workflow·Agent 중 어디에 속하는지 이유와 함께 반환된다.로 정의하는가

복구

모든 단계를 상위 모델에게 맡겨 같은 입력에도 비용과 결과가 달라진다. 발생 시
01_architecture.json에서 원인 상태를 확인

안전

회의 Agent 전체 지도의 사람 승인과
external_write=false 위치

관측

01_architecture.json에서 어떤 status·error_code를
보는가

01_architecture.json 실행 증거

- 01 — 설명: 회의 Agent 전체 지도이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py`
- 03 — 증거: 01_architecture.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 세 가지 입력과 STT 선택

DAY 2 · 2차시

09:50-10:40 (쉬는 시간·점심시간 11:30-13:00)

세 가지 입력과 STT 선택

Input adapter · STT · 02_inputs.json

진행

세 가지 입력과 STT 선택
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

data/day2_public_audio/sources.json
src/course_services/
day2_meeting_workflow.py

확인할 결과

02_inputs.json

2차시 입력과 결과

입력	——	01_architecture.json
이번 차시	——	세 가지 입력과 STT 선택
결과	——	02_inputs.json
다음 연결	——	도메인 맥락과 MCP 정책

세 가지 입력과 STT 선택 필요성

이미 있는 텍스트를 다시 STT하면 느리고 화자 정보도 잃는다.

입력 선택에서 02_inputs.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 02_inputs.json

세 가지 입력과 STT 선택 학습 결과

- 01 — Meet 전사·ClovaNote TXT·녹음 파일을 하나의 TranscriptEnvelope로 변환한다.
- 02 — 공개 MP3를 선택하고 실제로는 합성 fixture 전사를 결과처럼 보여준다. 왜 위험한지 설명한다.
- 03 — 02_inputs.json에서 정상과 실패 상태를 직접 확인한다.

세 가지 입력과 STT 선택 용어

용어	이 수업에서 쓰는 뜻
Input adapter	다른 형식을 공통 구조로 바꾸는 코드
STT	음성을 텍스트로 바꾸는 처리
Source mode	전사가 어디서 왔는지 남기는 필드
Provenance	출처·시각·변환 이력

02_inputs.json 입출력

INPUT

data/day2_public_audio/sources.json

OUTPUT

02_inputs.json

PROCESS

입력 선택 → 파서 또는 STT → 화자·시각 보존

VERIFY

세 입력이 각각 다른 source_mode와 segment evidence를 남긴다.
live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.

세 가지 입력과 STT 선택 실행 흐름

01

입력 선택

02

파서 또는 STT

03

화자·시각 보존

04

출처 기록

05

공통 계약 검증

마지막 판단은 02_inputs.json에서 사람이 확인합니다.

02_inputs.json 정상·실패 계약

- 정상 — 세 입력이 각각 다른 `source_mode`와 `segment evidence`를 남긴다.
- 경계 — live STT 실패는 다른 음성의 `fixture` 전사로 조용히 대체되지 않는다.
- 외부 쓰기 — 세 가지 입력과 STT 선택: 기본값 `false` · `dry-run`과 사람 승인 뒤 별도 `adapter` 호출
- 기록 — `status·error_code·입력 식별자·02_inputs.json`

세 가지 입력과 STT 선택 파일 지도

입력·fixture

data/day2_public_audio/sources.json

구현 코드

src/course_services/day2_meeting_workflow.py

검증·test

scripts/day2_public_audio.py

따라하기 notebook

output/course-labs/day2-v2/02_inputs.json

세 가지 입력과 STT 선택 개념·코드

Input adapter

다른 형식을 공통 구조로 바꾸는 코드

```
data/day2_public_audio/sources.json
```

STT

음성을 텍스트로 바꾸는 처리

```
src/course_services/day2_meeting_workflow.py
```

Source mode

전사가 어디서 왔는지 남기는 필드

```
scripts/day2_public_audio.py
```

Provenance

출처·시각·변환 이력

```
output/course-labs/day2-v2/02_inputs.json
```


세 가지 입력과 STT 선택 선택 기준

구분	역할	이번 차시의 판단 기준
Input adapter	입력 선택 단계의 입력 기준	결정론 test로 먼저 확인
STT	파서 또는 STT 단계의 교체 경계	adapter 경계를 확인
Source mode	화자·시각 보존 단계의 운영 조건	비용·지연·권한을 확인
Provenance	출처 기록 단계의 판단 기록	사람 판단과 운영 기록을 확인

세 가지 입력과 STT 선택 대표 실패

실패

공개 MP3를 선택하고 실제로는 합성 fixture 전사를 결과처럼 보여준다.

사용자 영향

공개 MP3를 선택하고 실제로는 합성 fixture 전사를 결과처럼 보여준다. 상태에서는 02_inputs.json을 운영 판단에 사용할 수 없다.

세 가지 입력과 STT 선택 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

세 가지 입력과 STT 선택 복구 경로

- 01 — 탐지: live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.
- 02 — 중단: 공개 MP3를 선택하고 실제로는 합성 fixture 전사를 결과처럼 보여준다.
- 03 — 복구: public live STT와 fixture WAV·TXT 쌍을 다른 source_mode로 남기고 섞지 않는다.
- 04 — 증거: 02_inputs.json과 test 결과를 함께 남긴다.

세 가지 입력과 STT 선택 Codex 검증 루프

src/course_services/
day2_meeting_workflow.py의
책임을 찾고 02_inputs.json
계약을 focused test로
고정합니다.

세 가지 입력과 STT 선택
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



세 가지 입력과 STT 선택 Codex 작업 명세

목표

세 adapter의 정상·빈 TXT·손상 음성·source 혼용 test를 작성한다.

허용 경로

src/course_services/day2_meeting_workflow.py
scripts/day2_public_audio.py

완료 조건

세 입력이 각각 다른 source_mode와 segment evidence를 남긴다.
live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.

금지

세 가지 입력과 STT 선택: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

세 가지 입력과 STT 선택 독립 리뷰

- | | | | |
|----|---------------|----|---|
| 01 | Codex diff | —— | 세 가지 입력과 STT 선택 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 세 입력이 각각 다른 source_mode와 segment evidence를 남긴다. |
| 03 | Claude review | —— | 02_inputs.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 세 가지 입력과 STT 선택의 두 LLM 의견과 test는 자동 승인이 아니다 |

세 가지 입력과 STT 선택 정상 시연

- 01 — 열기: `data/day2_public_audio/sources.json`
- 02 — 구현 확인: `src/course_services/day2_meeting_workflow.py`
- 03 — 실행: `python scripts/day2_public_audio.py resolve --path-only`
- 04 — 성공 신호: 세 입력이 각각 다른 `source_mode`와 `segment evidence`를 남긴다.

세 가지 입력과 STT 선택 실행 명령

```
$ python scripts/day2_public_audio.py resolve --path-only
```

실행 전 확인

세 가지 입력과 STT 선택: repository root · Python 3.12 · .env 미출력

02_inputs.json 실행 결과

```
{
  "contracts": {
    "google_meet_text": {
      "source_mode": "google_meet_text",
      "source_count": 1,
      "segment_count": 4,
      "first_segment": {
        "id": "s01",
        "speaker": "민지",
        "text": "오늘은 배송 자연 회의 기록 자동화 범위를 확정하겠습니다.",
        "start_seconds": 0,
        "quality_flags": []
      },
      "stt_metadata": null
    },
    "clovanote_txt": {
      "source_mode": "clovanote_txt",
      "source_count": 1,
      "segment_count": 3,
      "first_segment": {
        "id": "s01",
        "speaker": "민지",
        "text": "배송 자연 원인 분류를 1차 범위로 확정합니다.",
        "start_seconds": 0,
        "quality_flags": []
      },
      ...
    }
  }
}
```

세 가지 입력과 STT 선택 실패·복구 시연

재현

live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.

복구

public live STT와 fixture WAV·TXT 쌍을 다른 source_mode로 남기고 섞지 않는다.

확인: live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.

세 가지 입력과 STT 선택 소프트웨어 제작

열 파일	<code>src/course_services/day2_meeting_workflow.py</code> <code>scripts/day2_public_audio.py</code>
작업 범위	세 가지 입력과 STT 선택 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	세 입력이 각각 다른 <code>source_mode</code> 와 <code>segment evidence</code> 를 남긴다. <code>02_inputs.json</code> 저장

세 가지 입력과 STT 선택 Notebook 셀

materials/day2/day2_service_lab.ipynb · 2차시

```
from src.meeting_demo import parse_transcript

meet_text = "\n".join([
    "[00:00] 민지: 오늘은 배송 지연 회의 기록 자동화 범위를 확정하겠습니다.",
    "[00:18] 준호: WISMO 문의를 우선 처리하고 환불 자동화는 보류하는 것이 좋겠습니다.",
    "[00:37] 서연: 제가 9월 2일까지 고객 안내 문구를 정리해 공유하겠습니다.",
    "[00:55] 민지: 최근 야근 부담이 있으니 이번 범위를 더 늘리지 않겠습니다.",
])
clova_text = "\n".join([
    "화자 1 00:00", "배송 지연 원인 분류를 1차 범위로 확정합니다.",
    "화자 2 00:24", "제가 9월 3일까지 분류 기준을 작성하겠습니다.",
    "화자 1 00:46", "운영팀 부담을 확인한 뒤 다음 범위를 결정하겠습니다.",
])
audio_path = ROOT / "data/meeting_sample_ko_12min.wav"

sources = {
    "google_meet_text": SourceInput(
        source_mode="google_meet_text", source_ref="meet://fixture/delivery-001",
        meet_transcript=meet_text,
        speaker_metadata={
# ... 다음 줄은 Notebook에서 계속
```

세 가지 입력과 STT 선택 Terminal 실행

```
$ python scripts/day2_public_audio.py resolve --path-only
```

저장 위치

```
output/course-labs/day2-v2/02_inputs.json
```

세 입력이 각각 다른 source_mode와 segment evidence를 남긴다.
실패 시 02_inputs.json의 status:error_code를 먼저 확인

세 가지 입력과 STT 선택 실패 증거

live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.

```
result_file = "output/course-labs/day2-v2/02_inputs.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

02_inputs.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

세 가지 입력과 STT 선택 정상 Case

NORMAL

세 입력이 각각 다른 `source_mode`와 `segment evidence`를 남긴다.

- 01 — data/day2_public_audio/sources.json 입력과 command가 기록됐는가
- 02 — 02_inputs.json schema가 validate되는가
- 03 — 세 가지 입력과 STT 선택 focused test가 실제로 통과했는가

세 가지 입력과 STT 선택 경계 Case

BOUNDARY

live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다.

- 01 — 세 가지 입력과 STT 선택: raw traceback 대신 stable error_code가 남는가
- 02 — 02_inputs.json: 외부 쓰기·자동 메일이 false인가
- 03 — live STT 실패는 다른 음성의 fixture 전사로 조용히 대체되지 않는다. 재현이 가능한가

세 가지 입력과 STT 선택 현업 적용

회의 도구가 달라도 같은 결과 계약을 쓰는 입력층

- 01 — 세 가지 입력과 STT 선택은 합성·비식별 sample로 먼저 실행
- 02 — 회의 도구가 달라도 같은 결과 계약을 쓰는 입력층의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 02_inputs.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 02_inputs.json을 운영 검토 자료로 사용

세 가지 입력과 STT 선택 포트폴리오 적용

세 가지 입력과 STT 선택 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 공개 MP3를 선택하고 실제로는 합성 fixture 전사를 결과처럼 보여준다.의 사용자 영향을 한 문장으로 설명
- 02 — 세 가지 입력과 STT 선택 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 세 adapter의 정상·빈 TXT·손상 음성·source 혼용 test를 작성한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 02_inputs.json과 README 재현 절차를 제출

세 가지 입력과 STT 선택 운영 점검

품질

무엇을 세 입력이 각각 다른 source_mode와 segment evidence를 남긴다.로 정의하는가

복구

공개 MP3를 선택하고 실제로는 합성 fixture 전사를 결과처럼 보여준다. 발생 시 02_inputs.json에서 원인 상태를 확인

안전

세 가지 입력과 STT 선택의 사람 승인과 external_write=false 위치

관측

02_inputs.json에서 어떤 status·error_code를 보는가

02_inputs.json 실행 증거

- 01 — 설명: 세 가지 입력과 STT 선택이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python scripts/day2_public_audio.py resolve --path-only`
- 03 — 증거: 02_inputs.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 도메인 맥락과 MCP 정책

DAY 2 · 3차시

10:40-11:30 (쉬는 시간·점심시간 11:30-13:00)

도메인 맥락과 MCP 정책

Domain context · MCP · 03_domain_context.json

진행

도메인 맥락과 MCP 정책
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/
day2_meeting_workflow.py
materials/day2/Codex_Claude_대화_
시나리오.md

확인할 결과

03_domain_context.json

3차시 입력과 결과

입력	————	02_inputs.json
이번 차시	————	도메인 맥락과 MCP 정책
결과	————	03_domain_context.json
다음 연결	————	MeetingRecord 결과 계약

도메인 맥락과 MCP 정책 필요성

Notion·Confluence·Slack을 많이 읽는 것과 필요한 근거를 잘 찾는 것은 다르다.

도메인 설명에서 03_domain_context.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 03_domain_context.json

도메인 맥락과 MCP 정책 학습 결과

- 01 — 업무 도메인·기존 맥락·외부 자료 탐색 범위를 사용자가 통제한다.
- 02 — 회의명이 비슷하다는 이유로 전체 workspace와 장기 대화를 무제한으로 읽는다. 왜 위험한지 설명한다.
- 03 — 03_domain_context.json에서 정상과 실패 상태를 직접 확인한다.

도메인 맥락과 MCP 정책 용어

용어	이 수업에서 쓰는 뜻
Domain context	업계 용어·제품·의사결정 기준
MCP	Desktop AI가 외부 도구를 호출하는 연결 계약
Retrieval policy	무엇을 언제까지 읽을지의 규칙
Least privilege	필요한 읽기 권한만 주는 원칙

03_domain_context.json 입출력

INPUT

src/course_services/day2_meeting_workflow.py

OUTPUT

03_domain_context.json

PROCESS

도메인 설명 → 최근 범위 → 읽기 계획

VERIFY

각 근거에 source·time window·reason이 남고 write tool은 계획에 없다.
외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.

도메인 맥락과 MCP 정책 실행 흐름

01

도메인 설명

02

최근 범위

03

읽기 계획

04

근거 수집

05

민감정보 제외

마지막 판단은 03_domain_context.json에서 사람이 확인합니다.

03_domain_context.json 정상·실패 계약

정상	——	각 근거에 source·time window·reason이 남고 write tool은 계획에 없다.
경계	——	외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.
외부 쓰기	——	도메인 맥락과 MCP 정책: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·03_domain_context.json

도메인 맥락과 MCP 정책 파일 지도

입력·fixture

src/course_services/day2_meeting_workflow.py

구현 코드

materials/day2/Codex_Claude_대화_시나리오.md

검증·test

tests/test_day2_meeting_workflow.py

따라하기 notebook

output/course-labs/day2-v2/03_domain_context.json

도메인 맥락과 MCP 정책 개념·코드

Domain context

업계 용어·제품·의사결정 기준

```
src/course_services/day2_meeting_workflow.py
```

Retrieval policy

무엇을 언제까지 읽을지의 규칙

```
tests/test_day2_meeting_workflow.py
```

MCP

Desktop AI가 외부 도구를 호출하는 연결 계약

```
materials/day2/Codex_Claude_대화_시나리오.md
```

Least privilege

필요한 읽기 권한만 주는 원칙

```
output/course-labs/day2-v2/03_domain_context.json
```

도메인 맥락과 MCP 정책 선택 기준

구분	역할	이번 차시의 판단 기준
Domain context	도메인 설명 단계의 입력 기준	결정론 test로 먼저 확인
MCP	최근 범위 단계의 교체 경계	adapter 경계를 확인
Retrieval policy	읽기 계획 단계의 운영 조건	비용·지연·권한을 확인
Least privilege	근거 수집 단계의 판단 기록	사람 판단과 운영 기록을 확인

도메인 맥락과 MCP 정책 대표 실패

실패

회의명이 비슷하다는 이유로 전체 workspace와 장기 대화를 무제한으로 읽는다.

사용자 영향

회의명이 비슷하다는 이유로 전체 workspace와 장기 대화를 무제한으로 읽는다.
상태에서는 03_domain_context.json을 운영 판단에 사용할 수 없다.

도메인 맥락과 MCP 정책 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

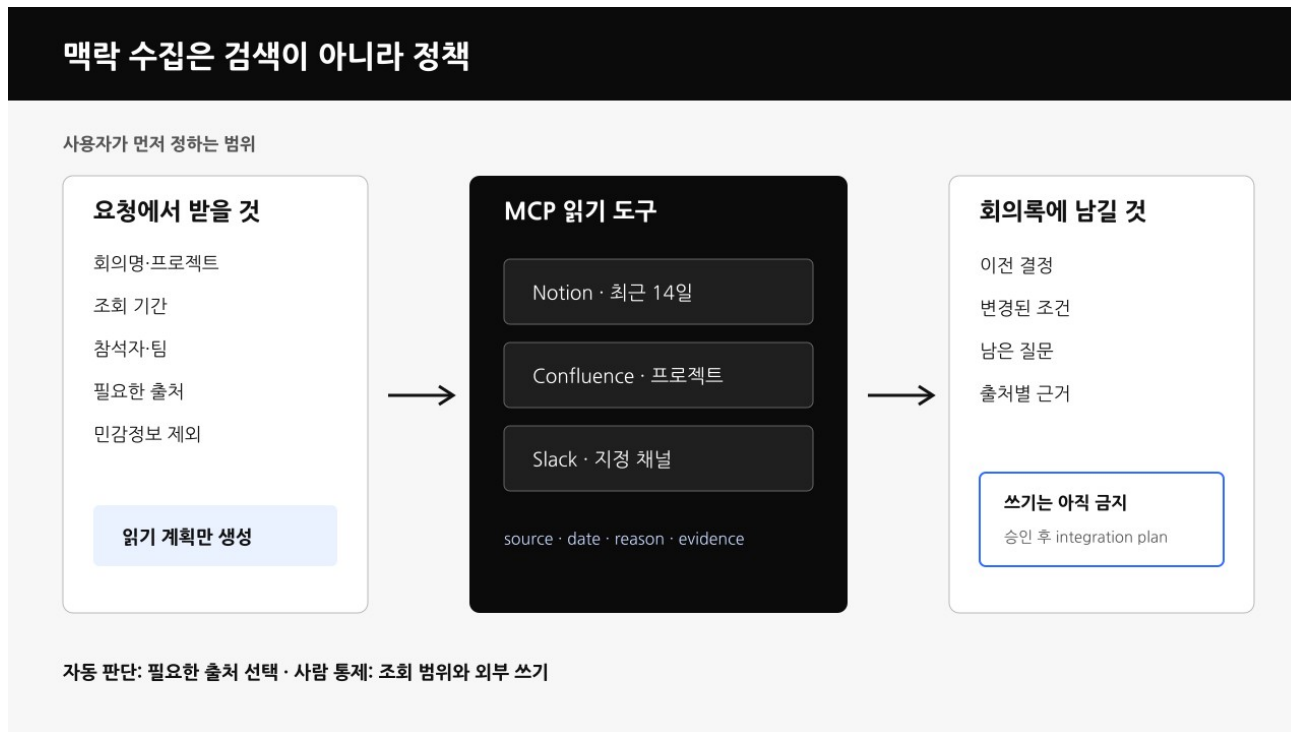
도메인 맥락과 MCP 정책 복구 경로

- 01 — 탐지: 외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.
- 02 — 중단: 회의명이 비슷하다는 이유로 전체 workspace와 장기 대화를 무제한으로 읽는다.
- 03 — 복구: 회의자·기간·프로젝트·출처를 제한하고 도구 계획을 사람이 확인한다.
- 04 — 증거: 03_domain_context.json과 test 결과를 함께 남긴다.

도메인 맥락과 MCP 정책 Codex 검증 루프

materials/day2/Codex_Claude_대화_시나리오.md의 책임을
찾고 03_domain_context.json
계약을 focused test로
고정합니다.

도메인 맥락과 MCP 정책
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



도메인 맥락과 MCP 정책 Codex 작업 명세

목표

요약에 필요한 MCP 읽기만 계획하고 쓰기 도구를 제외하는 policy test를 만든다.

허용 경로

materials/day2/Codex_Claude_대화_시나리오.md
tests/test_day2_meeting_workflow.py

완료 조건

각 근거에 source·time window·reason이 남고 write tool은 계획에 없다.
외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.

금지

도메인 맥락과 MCP 정책: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

도메인 맥락과 MCP 정책 독립 리뷰

- | | | | |
|----|---------------|----|---|
| 01 | Codex diff | —— | 도메인 맥락과 MCP 정책 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 각 근거에 source·time window·reason이 남고 write tool은 계획에 없다. |
| 03 | Claude review | —— | 03_domain_context.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 도메인 맥락과 MCP 정책의 두 LLM 의견과 test는 자동 승인이 아니다 |

도메인 맥락과 MCP 정책 정상 시연

- 01 — 열기: `src/course_services/day2_meeting_workflow.py`
- 02 — 구현 확인: `materials/day2/Codex_Claude_대화_시나리오.md`
- 03 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py -k context`
- 04 — 성공 신호: 각 근거에 `source·time window·reason`이 남고 `write tool`은 계획에 없다.

도메인 맥락과 MCP 정책 실행 명령

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py -k context
```

실행 전 확인

도메인 맥락과 MCP 정책: repository root · Python 3.12 · .env 미출력

03_domain_context.json 실행 결과

```
{
  "domain_context": {
    "industry": "이커머스 고객경험",
    "organization_context": "배송 지연 문의가 증가해 상담 부담과 고객 불편이 함께 커진 상태",
    "meeting_objective": "배송 지연 회의 기록 자동화 범위 확정",
    "glossary": {
      "WISMO": "배송 위치 문의",
      "SLA": "약속한 응답 시간"
    },
    "prior_decisions": [
      "외부 발송은 사람 승인 뒤에만 진행",
      "환불 자동화는 이번 범위에서 제외"
    ],
    "desired_outcomes": [
      "근거가 있는 담당자별 To Do",
      "단기·중기·장기 인사이트"
    ],
    "confidentiality": "internal"
  },
  "context_prompt_fields": {
    "industry": "이커머스 고객경험",
    "organization_context": "배송 지연 문의가 증가해 상담 부담과 고객 불편이 함께 커진 상태",
    "meeting_objective": "배송 지연 회의 기록 자동화 범위 확정",
    "glossary": {
      "WISMO": "배송 위치 문의",
    }
  },
  ...
}
```

도메인 맥락과 MCP 정책 실패·복구 시연

재현

외부 고객·비공개 채널·기간 밖 문서는
POLICY_BLOCKED다.

복구

회의자·기간·프로젝트·출처를 제한하고
도구 계획을 사람이 확인한다.

확인: 외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.

도메인 맥락과 MCP 정책 소프트웨어 제작

열 파일	<code>materials/day2/Codex_Claude_대화_시나리오.md</code> <code>tests/test_day2_meeting_workflow.py</code>
작업 범위	도메인 맥락과 MCP 정책 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	각 근거에 source·time window·reason이 남고 write tool은 계획에 없다. 03_domain_context.json 저장

도메인 맥락과 MCP 정책 Notebook 셀

materials/day2/day2_service_lab.ipynb · 3차시

```
domain_context = DomainContext(
    industry="이커머스 고객경험",
    organization_context="배송 지연 문의가 증가해 상담 부담과 고객 불편이 함께 커진 상태",
    meeting_objective="배송 지연 회의 기록 자동화 범위 확정",
    glossary={"WISMO": "배송 위치 문의", "SLA": "약속한 응답 시간"},
    prior_decisions=["외부 발송은 사람 승인 뒤에만 진행", "환불 자동화는 이번 범위에서 제외"],
    desired_outcomes=["근거가 있는 담당자별 To Do", "단기·중기·장기 인사이트"],
    confidentiality="internal",
)
retrieval_policy = MCPRetrievalPolicy(
    allowed_connectors=["notion", "confluence", "slack"],
    explicit_user_authorization=True,
    lookback_days=14,
    allowed_scopes={
        "notion": ["CX PoC"], "confluence": ["CX 정책"], "slack": ["#delivery-poc"],
    },
    participant_match_required=True,
    max_items_per_connector=12,
)
mcp_plan = build_mcp_retrieval_plan(
# ... 다음 줄은 Notebook에서 계속
```

도메인 맥락과 MCP 정책 Terminal 실행

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py -k context
```

저장 위치

```
output/course-labs/day2-v2/03_domain_context.json
```

각 근거에 source·time window·reason이 남고 write tool은 계획에 없다.
실패 시 03_domain_context.json의 status·error_code를 먼저 확인

도메인 맥락과 MCP 정책 실패 증거

외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.

```
result_file = "output/course-labs/day2-v2/03_domain_context.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

03_domain_context.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

도메인 맥락과 MCP 정책 정상 Case

NORMAL

각 근거에 source·time window·reason이 남고 write tool은 계획에 없다.

- 01 — src/course_services/day2_meeting_workflow.py 입력과 command가 기록됐는가
- 02 — 03_domain_context.json schema가 validate되는가
- 03 — 도메인 맥락과 MCP 정책 focused test가 실제로 통과했는가

도메인 맥락과 MCP 정책 경계 Case

BOUNDARY

외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다.

- 01 — 도메인 맥락과 MCP 정책: raw traceback 대신 stable error_code가 남는가
- 02 — 03_domain_context.json: 외부 쓰기·자동 메일이 false인가
- 03 — 외부 고객·비공개 채널·기간 밖 문서는 POLICY_BLOCKED다. 재현이 가능한가

도메인 맥락과 MCP 정책 현업 적용

회의 전후의 결정 배경만 찾아 맥락을 복원하는 Desktop Agent

- 01 — 도메인 맥락과 MCP 정책은 합성·비식별 sample로 먼저 실행
- 02 — 회의 전후의 결정 배경만 찾아 맥락을 복원하는 Desktop Agent의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 03_domain_context.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 03_domain_context.json을 운영 검토 자료로 사용

도메인 맥락과 MCP 정책 포트폴리오 적용

도메인 맥락과 MCP 정책 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 회의명이 비슷하다는 이유로 전체 workspace와 장기 대화를 무제한으로 읽는다.의 사용자 영향을 한 문장으로 설명
- 02 — 도메인 맥락과 MCP 정책 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 요약에 필요한 MCP 읽기만 계획하고 쓰기 도구를 제외하는 policy test를 만든다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 03_domain_context.json과 README 재현 절차를 제출

도메인 맥락과 MCP 정책 운영 점검

품질

무엇을 각 근거에 source·time window·reason이
남고 write tool은 계획에 없다.로 정의하는가

복구

회의명이 비슷하다는 이유로 전체 workspace와
장기 대화를 무제한으로 읽는다. 발생 시
03_domain_context.json에서 원인 상태를 확인

안전

도메인 맥락과 MCP 정책의 사람 승인과
external_write=false 위치

관측

03_domain_context.json에서 어떤
status·error_code를 보는가

03_domain_context.json 실행 증거

- 01 — 설명: 도메인 맥락과 MCP 정책이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py -k context`
- 03 — 증거: 03_domain_context.json과 정상·실패 test 결과가 남아 있다.

다음 차시: MeetingRecord 결과 계약

DAY 2 · 4차시

13:00-13:50 (쉬는 시간 14:40-15:00)

MeetingRecord 결과 계약

Schema · Evidence · 04_strategy_compare.json

진행

MeetingRecord 결과 계약
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/
day2_meeting_workflow.py
tests/test_day2_meeting_workflow.py

확인할 결과

04_strategy_compare.json

4차시 입력과 결과

입력	————	03_domain_context.json
이번 차시	————	MeetingRecord 결과 계약
결과	————	04_strategy_compare.json
다음 연결	————	Codex·Claude 시나리오 설계

MeetingRecord 결과 계약 필요성

담당자·기한·감정 상태를 추측하면 회의록이 아니라 조직 리스크가 된다.

목적·맥락에서 04_strategy_compare.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 04_strategy_compare.json

MeetingRecord 결과 계약 학습 결과

- 01 — 목적·이전 맥락·요약·참석자 관점·To Do·단중장기 통찰을 근거와 구조화한다.
- 02 — 원문에 없는 owner·due date·개인의 생각을 그럴듯하게 보완한다. 왜 위험한지 설명한다.
- 03 — 04_strategy_compare.json에서 정상과 실패 상태를 직접 확인한다.

MeetingRecord 결과 계약 용어

용어	이 수업에서 쓰는 뜻
Schema	결과 필드와 형식의 약속
Evidence	결과를 뒷받침하는 원문 구간
Nullable	확실하지 않은 값을 비워 두는 설계
Well-being	부하·갈등·미해결을 단정 없이 표시하는 검토 관점

04_strategy_compare.json 입출력

INPUT

src/course_services/day2_meeting_workflow.py

OUTPUT

04_strategy_compare.json

PROCESS

목적·맥락 → 발화자별 관점 → 결정·To Do

VERIFY

회의 목적부터 단중장기 통찰까지 evidence와 함께
validate된다.
근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.

MeetingRecord 결과 계약 실행 흐름

01

목적·맥락

02

발화자별 관점

03

결정·To Do

04

단중장기 통찰

05

evidence·null 검증

마지막 판단은 04_strategy_compare.json에서 사람이 확인합니다.

04_strategy_compare.json 정상·실패 계약

정상	—— 회의 목적부터 단중장기 통찰까지 evidence와 함께 validate된다.
경계	—— 근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.
외부 쓰기	—— MeetingRecord 결과 계약: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·04_strategy_compare.json

MeetingRecord 결과 계약 파일 지도

입력·fixture

src/course_services/day2_meeting_workflow.py

구현 코드

tests/test_day2_meeting_workflow.py

검증·test

materials/day2/day2_service_lab.ipynb

따라하기 notebook

output/course-labs/day2-v2/04_strategy_compare.json

MeetingRecord 결과 계약 개념·코드

Schema

결과 필드와 형식의 약속

```
src/course_services/day2_meeting_workflow.py
```

Evidence

결과를 뒷받침하는 원문 구간

```
tests/test_day2_meeting_workflow.py
```

Nullable

확실하지 않은 값을 비워 두는 설계

```
materials/day2/day2_service_lab.ipynb
```

Well-being

부하·갈등·미해결을 단정 없이 표시하는 검토 관점

```
output/course-labs/day2-v2/04_strategy_compare.json
```

MeetingRecord 결과 계약 선택 기준

구분	역할	이번 차시의 판단 기준
Schema	목적·맥락 단계의 입력 기준	결정론 test로 먼저 확인
Evidence	발화자별 관점 단계의 교체 경계	adapter 경계를 확인
Nullable	결정·To Do 단계의 운영 조건	비용·지연·권한을 확인
Well-being	단중장기 통찰 단계의 판단 기록	사람 판단과 운영 기록을 확인

MeetingRecord 결과 계약 대표 실패

실패

원문에 없는 owner·due date·개인의 생각을 그럴듯하게 보완한다.

사용자 영향

원문에 없는 owner·due date·개인의 생각을 그럴듯하게 보완한다. 상태에서는 04_strategy_compare.json을 운영 판단에 사용할 수 없다.

MeetingRecord 결과 계약 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

MeetingRecord 결과 계약 복구 경로

- 01 — 탐지: 근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.
- 02 — 중단: 원문에 없는 owner·due date·개인의 생각을 그럴듯하게 보완한다.
- 03 — 복구: 미상값은 null·CONFIRM_REQUIRED로 두고 사실 필드에 evidence ID를 요구한다.
- 04 — 증거: 04_strategy_compare.json과 test 결과를 함께 남긴다.

MeetingRecord 결과 계약 Codex 검증 루프

tests/
test_day2_meeting_workflow.py의
책임을 찾고
04_strategy_compare.json 계약을
focused test로 고정합니다.

MeetingRecord 결과 계약
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

MeetingRecord · 사람이 확인할 결과 계약

```
{  
  "purpose": "이 회의가 풀려는 문제",  
  "prior_context": ["이전 결정 + source"],  
  "summary": { "text": "...", "evidence_ids": ["s003"] },  
  "participant_perspectives": [{ "speaker": "...", "evidence_ids": [...] }],  
  "action_items": [{ "owner": null, "due": null, "status": "CONFIRM_REQUIRED" }],  
  "insights": { "short": [...], "mid": [...], "long": [...] },  
  "wellbeing_signals": [{ "observation": "...", "diagnosis": false }],  
  "review": { "decision": "approve | edit | reject" },  
  "external_write": false  
}
```

근거 없으면 추측하지 않기
null · CONFIRM_REQUIRED · HOLD

톤은 부드럽게, 사실은 보수적으로, 외부 반영은 승인 후에만

MeetingRecord 결과 계약 Codex 작업 명세

목표

MeetingRecord schema와 근거 누락·unknown owner·extra field test를 함께 만든다.

허용 경로

tests/test_day2_meeting_workflow.py
materials/day2/day2_service_lab.ipynb

완료 조건

회의 목적부터 단종장기 통찰까지 evidence와 함께 validate된다.
근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.

금지

MeetingRecord 결과 계약: secret 출력 · workspace 밖 변경 · test 악화 · 승인 없는 외부 쓰기

MeetingRecord 결과 계약 독립 리뷰

- | | | | |
|----|---------------|----|---|
| 01 | Codex diff | —— | MeetingRecord 결과 계약 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 회의 목적부터 단증장기 통찰까지 evidence와 함께 validate된다. |
| 03 | Claude review | —— | 04_strategy_compare.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | MeetingRecord 결과 계약의 두 LLM 의견과 test는 자동 승인이 아니다 |

MeetingRecord 결과 계약 정상 시연

- 01 — 열기: `src/course_services/day2_meeting_workflow.py`
- 02 — 구현 확인: `tests/test_day2_meeting_workflow.py`
- 03 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py -k record`
- 04 — 성공 신호: 회의 목적부터 단종장기 통찰까지 evidence와 함께 validate된다.

MeetingRecord 결과 계약 실행 명령

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py -k record
```

실행 전 확인

MeetingRecord 결과 계약: repository root · Python 3.12 · .env 미출력

04_strategy_compare.json 실행 결과

```
{
  "comparison": [
    {
      "strategy": "single_llm",
      "control_flow": "한 번의 구조화 요청",
      "best_for": "입력과 산출물이 단순한 일회성 작업",
      "llm_calls": "1",
      "strength": "가장 빠른 시작",
      "risk": "누락·형식 변동을 한 호출 안에서 발견하기 어려움"
    },
    {
      "strategy": "deterministic_workflow",
      "control_flow": "고정 노드와 검증 규칙",
      "best_for": "STT→요약→근거→승인처럼 순서가 정해진 반복 업무",
      "llm_calls": "0~필요한 노드 수",
      "strength": "비용·오류 위치·재실행 범위 예측 가능",
      "risk": "새 유즈케이스는 사람이 Workflow를 추가해야 함"
    },
    {
      "strategy": "agent_router",
      "control_flow": "요청·정책을 읽고 도구/Workflow 선택",
      "best_for": "Notion·Confluence·Slack 등 필요한 정보원이 요청마다 달라지는 작업",
      "llm_calls": "상황별 다수",
      "strength": "모호한 요청과 다양한 도구 조합에 대응",
      "risk": "비용·권한·잘못된 도구 선택 위험이 가장 큼"
    }
  ]
}
```

MeetingRecord 결과 계약 실패·복구 시연

재현

근거 없는 개인 성향 판단·진단·자동
이메일은 차단된다.

복구

미상값은 null·CONFIRM_REQUIRED로
두고 사실 필드에 evidence ID를
요구한다.

확인: 근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.

MeetingRecord 결과 계약 소프트웨어 제작

열 파일

tests/test_day2_meeting_workflow.py
materials/day2/day2_service_lab.ipynb

작업 범위

MeetingRecord 결과 계약 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

회의 목적부터 단종장기 통찰까지 evidence와 함께 validate된다.
04_strategy_compare.json 저장

MeetingRecord 결과 계약 Notebook 셀

materials/day2/day2_service_lab.ipynb · 4차시

```
strategy_table = compare_execution_strategies()
routing_cases = {
    "fixed_meeting_record": route_execution_strategy(
        requested_actions=["normalize", "summarize", "perspectives", "todos", "insights", "draft"]
    ),
    "context_retrieval_needed": route_execution_strategy(
        requested_actions=["summarize", "todos"],
        external_context_sources=["notion", "confluence", "slack"],
    ),
    "one_off_unmodeled_request": route_execution_strategy(
        requested_actions=["rewrite_as_podcast_script"]
    ),
}
cost_guard = {
    "default_router_llm_calls": 0,
    "single_llm_expected_calls": 1,
    "deterministic_workflow_default_llm_calls": 0,
    "agent_router_budget_must_be_set": True,
    "stop_conditions": ["MAX_TOOL_CALLS", "MAX_TOKEN_BUDGET", "POLICY_DENIED", "HUMAN_REVIEW"],
}
# ... 다음 줄은 Notebook에서 계속
```


MeetingRecord 결과 계약 Terminal 실행

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py -k record
```

저장 위치

output/course-labs/day2-v2/04_strategy_compare.json

회의 목적부터 단중장기 통찰까지 evidence와 함께 validate된다.
실패 시 04_strategy_compare.json의 status:error_code를 먼저 확인

MeetingRecord 결과 계약 실패 증거

근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.

```
result_file = "output/course-labs/day2-v2/04_strategy_compare.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

04_strategy_compare.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

MeetingRecord 결과 계약 정상 Case

NORMAL

회의 목적부터 단증장기 통찰까지 evidence와 함께 validate된다.

- 01 — src/course_services/day2_meeting_workflow.py 입력과 command가 기록됐는가
- 02 — 04_strategy_compare.json schema가 validate되는가
- 03 — MeetingRecord 결과 계약 focused test가 실제로 통과했는가

MeetingRecord 결과 계약 경계 Case

BOUNDARY

근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다.

- 01 — MeetingRecord 결과 계약: raw traceback 대신 stable error_code가 남는가
- 02 — 04_strategy_compare.json: 외부 쓰기·자동 메일이 false인가
- 03 — 근거 없는 개인 성향 판단·진단·자동 이메일은 차단된다. 재현이 가능한가

MeetingRecord 결과 계약 현업 적용

판단·미해결·부하를 함께 보여주는 Well-being 회의기록

- 01 — MeetingRecord 결과 계약은 합성·비식별 sample로 먼저 실행
- 02 — 판단·미해결·부하를 함께 보여주는 Well-being 회의기록의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 04_strategy_compare.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 04_strategy_compare.json을 운영 검토 자료로 사용

MeetingRecord 결과 계약 포트폴리오 적용

MeetingRecord 결과 계약 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 원문에 없는 owner·due date·개인의 생각을 그럴듯하게 보완한다.의 사용자 영향을 한 문장으로 설명
- 02 — MeetingRecord 결과 계약 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — MeetingRecord schema와 근거 누락·unknown owner·extra field test를 함께 만든다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 04_strategy_compare.json과 README 재현 절차를 제출

MeetingRecord 결과 계약 운영 점검

품질

무엇을 회의 목적부터 단종장기 통찰까지 evidence와 함께 validate된다.로 정의하는가

복구

원문에 없는 owner·due date·개인의 생각을 그럴듯하게 보완한다. 발생 시 04_strategy_compare.json에서 원인 상태를 확인

안전

MeetingRecord 결과 계약의 사람 승인과 external_write=false 위치

관측

04_strategy_compare.json에서 어떤 status·error_code를 보는가

04_strategy_compare.json 실행 증거

- 01 — 설명: MeetingRecord 결과 계약이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py -k record`
- 03 — 증거: 04_strategy_compare.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Codex·Claude 시나리오 설계

DAY 2 · 5차시

13:50-14:40 (쉬는 시간 14:40-15:00)

Codex·Claude 시나리오 설계

Scenario tree · Harness · 05_workflow_runs.json

진행

Codex·Claude 시나리오 설계
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

materials/day2/Codex_Claude_대화_
시나리오.md
src/course_services/
day2_meeting_workflow.py

확인할 결과

05_workflow_runs.json

5차시 입력과 결과

입력 ————— 04_strategy_compare.json

이번 차시 ————— Codex·Claude 시나리오 설계

결과 ————— 05_workflow_runs.json

다음 연결 ————— LLM Provider와 비용 경계

Codex·Claude 시나리오 설계 필요성

상위 모델은 설계 파트너이지 통제를 넘길 대상이 아니다.

시나리오 3개에서 05_workflow_runs.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 05_workflow_runs.json

Codex·Claude 시나리오 설계 학습 결과

- 01 — 단계별 대화와 한 번의 Harness 요청을 비교하고 여러 시나리오를 test로 고정한다.
- 02 — '알아서 Agent를 만들어줘'라고만 요청해 권한·비용·실패 경계가 사라진다. 왜 위험한지 설명한다.
- 03 — 05_workflow_runs.json에서 정상과 실패 상태를 직접 확인한다.

Codex·Claude 시나리오 설계 용어

용어	이 수업에서 쓰는 뜻
Scenario tree	입력·목적·실패에 따른 여러 경로
Harness	목표·허용 범위·test·금지 행동을 함께 준 작업 계약
Codex	코드 수정·test·리뷰를 함께 하는 개발 Agent
Independent review	다른 session이 누락과 경계를 다시 보는 검토

05_workflow_runs.json 입출력

INPUT

materials/day2/Codex_Claude_대화_시나리오.md

OUTPUT

05_workflow_runs.json

PROCESS

시나리오 3개 → 완료 조건 → Codex 최소 diff

VERIFY

세 시나리오가 같은 결과 계약과 다른 입력 경로를
사용한다.
test 미실행.env 변경·외부 게시 추가 중 하나라도
있으면 HOLD다.

Codex·Claude 시나리오 설계 실행 흐름

01

시나리오 3개

02

완료 조건

03

Codex 최소 diff

04

Claude 독립 리뷰

05

사람 merge

마지막 판단은 05_workflow_runs.json에서 사람이 확인합니다.

05_workflow_runs.json 정상·실패 계약

정상	—— 세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다.
경계	—— test 미실행·env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다.
외부 쓰기	—— Codex·Claude 시나리오 설계: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·05_workflow_runs.json

Codex·Claude 시나리오 설계 파일 지도

입력·fixture

materials/day2/Codex_Claude_대화_시나리오.md

구현 코드

src/course_services/day2_meeting_workflow.py

검증·test

tests/test_day2_meeting_workflow.py

따라하기 notebook

output/course-labs/day2-v2/05_workflow_runs.json

Codex·Claude 시나리오 설계 개념·코드

Scenario tree

입력·목적·실패에 따른 여러 경로

```
materials/day2/Codex_Claude_대화_시나리오.md
```

Harness

목표·허용 범위·test·금지 행동을 함께 준 작업 계약

```
src/course_services/day2_meeting_workflow.py
```

Codex

코드 수정·test·리뷰를 함께 하는 개발 Agent

```
tests/test_day2_meeting_workflow.py
```

Independent review

다른 session이 누락과 경계를 다시 보는 검토

```
output/course-labs/day2-v2/05_workflow_runs.json
```

Codex·Claude 시나리오 설계 선택 기준

구분	역할	이번 차시의 판단 기준
Scenario tree	시나리오 3개 단계의 입력 기준	결정론 test로 먼저 확인
Harness	완료 조건 단계의 교체 경계	adapter 경계를 확인
Codex	Codex 최소 diff 단계의 운영 조건	비용·지연·권한을 확인
Independent review	Claude 독립 리뷰 단계의 판단 기록	사람 판단과 운영 기록을 확인

Codex·Claude 시나리오 설계 대표 실패

실패

'알아서 Agent를 만들어줘'라고만
요청해 권한·비용·실패 경계가
사라진다.

사용자 영향

'알아서 Agent를 만들어줘'라고만 요청해
권한·비용·실패 경계가 사라진다. 상태에서는
05_workflow_runs.json을 운영 판단에
사용할 수 없다.

Codex·Claude 시나리오 설계 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

Codex·Claude 시나리오 설계 복구 경로

- 01 — 탐지: test 미실행.env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다.
- 02 — 중단: '알아서 Agent를 만들어줘'라고만 요청해 권한·비용·실패 경계가 사라진다.
- 03 — 복구: 한 턴에는 한 책임만 주고 정상·경계 test 통과 후 범위를 늘린다.
- 04 — 증거: 05_workflow_runs.json과 test 결과를 함께 남긴다.

Codex·Claude 시나리오 설계 Codex 검증 루프

src/course_services/
day2_meeting_workflow.py의
책임을 찾고
05_workflow_runs.json 계약을
focused test로 고정합니다.

Codex·Claude 시나리오 설계
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

Codex와 대화하는 순서

01 · 상황

Meet·ClovaNote·녹음 세 입력을 지원하려고 해.

02 · 작은 목표

TranscriptEnvelope과 정상·빈 TXT test만 먼저 만들어줘.

03 · 경계

fixture와 live STT가 섞이지 않는 실패 test를 추가해줘.

04 · 독립 리뷰 → 사람 merge

Claude Code는 누락 경계와 변경 라인만 다시 검토·LLM 의견과 통과 test는 자동 승인이 아님

CODEX · 먼저 확인

현재 코드·test·입력 계약을 먼저 찾을게요.

CODEX · DIFF + TEST

허용 파일 2개만 변경하고 focused test를 실행했어요.

CODEX · EXPECTED FAILURE

SOURCE_MODE_MISMATCH로 HOLD되고 traceback은 노출하지 않

Codex·Claude 시나리오 설계 Codex 작업 명세

목표

Meet·ClovaNote·audio와 미승인 외부 쓰기 차단 test를 먼저 요청한다.

허용 경로

src/course_services/day2_meeting_workflow.py
tests/test_day2_meeting_workflow.py

완료 조건

세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다.
test 미실행·env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다.

금지

Codex·Claude 시나리오 설계: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Codex·Claude 시나리오 설계 독립 리뷰

- | | | | |
|----|---------------|----|--|
| 01 | Codex diff | —— | Codex·Claude 시나리오 설계 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다. |
| 03 | Claude review | —— | 05_workflow_runs.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Codex·Claude 시나리오 설계의 두 LLM 의견과 test는 자동 승인이 아니다 |

Codex·Claude 시나리오 설계 정상 시연

- 01 — 열기: `materials/day2/Codex_Claude_대화_시나리오.md`
- 02 — 구현 확인: `src/course_services/day2_meeting_workflow.py`
- 03 — 실행: `git diff -- src/course_services/day2_meeting_workflow.py
tests/test_day2_meeting_workflow.py`
- 04 — 성공 신호: 세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다.

Codex·Claude 시나리오 설계 실행 명령

```
$ git diff -- src/course_services/day2_meeting_workflow.py  
tests/test_day2_meeting_workflow.py
```

실행 전 확인

Codex·Claude 시나리오 설계: repository root · Python 3.12 · .env 미출력

05_workflow_runs.json 실행 결과

```
{
  "scenarios": {
    "google_meet_text": {
      "status": "DRAFT_READY",
      "source_mode": "google_meet_text",
      "segment_count": 4,
      "purpose": "배송 지연 회의 기록 자동화 범위 확정",
      "summary": "오늘은 배송 지연 회의 기록 자동화 범위를 확정하겠습니다. WISMO 문의를 우선 처리하고 환불 자동화는 보류하는 것이 좋겠습니다.",
      "participant_count": 3,
      "todo_count": 1,
      "wellbeing_risk_count": 1,
      "review": {
        "decision": "approve",
        "status": "APPROVED_READY_FOR_DRAFT",
        "export_ready": true,
        "human_reviewed": true,
        "external_write": false
      },
      "trace": [
        {
          "node": "policy",
          "status": "SUCCESS",
          "external_write": false,
          "source_mode": "google_meet_text",
          "source_count": 1,
          ...
        }
      ]
    }
  }
}
```

Codex·Claude 시나리오 설계 실패·복구 시연

재현

test 미실행·env 변경·외부 게시 추가 중
하나라도 있으면 HOLD다.

복구

한 턴에는 한 책임만 주고 정상·경계
test 통과 후 범위를 늘린다.

확인: test 미실행·env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다.

Codex·Claude 시나리오 설계 소프트웨어 제작

열 파일	<code>src/course_services/day2_meeting_workflow.py</code> <code>tests/test_day2_meeting_workflow.py</code>
작업 범위	Codex·Claude 시나리오 설계 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다. 05_workflow_runs.json 저장

Codex·Claude 시나리오 설계 Notebook 셀

materials/day2/day2_service_lab.ipynb · 5차시

```
workflow_runs = {
    "google_meet_text": run_meeting_workflow(
        sources["google_meet_text"], domain_context,
        review_decision="approve", retrieval_policy=retrieval_policy,
    ),
    "clovanote_txt": run_meeting_workflow(
        sources["clovanote_txt"], domain_context,
        review_decision="approve",
    ),
    "audio_stt": run_meeting_workflow(
        sources["audio_stt"], domain_context,
        review_decision="edit",
        review_edits={
            "meeting_summary": "고객 문의 자동화 PoC의 범위·금지 행동·담당자별 후속 조치를 근거와 함께 정리했습니다."
        },
        transcriber=reviewed_fixture_stt,
    ),
}
workflow_result = {
    "scenarios": {
# ... 다음 줄은 Notebook에서 계속
```

Codex·Claude 시나리오 설계 Terminal 실행

```
$ git diff -- src/course_services/day2_meeting_workflow.py  
tests/test_day2_meeting_workflow.py
```

저장 위치

output/course-labs/day2-v2/05_workflow_runs.json

세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다.
실패 시 05_workflow_runs.json의 status:error_code를 먼저 확인

Codex·Claude 시나리오 설계 실패 증거

test 미실행·.env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다.

```
result_file = "output/course-labs/day2-v2/05_workflow_runs.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

05_workflow_runs.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Codex·Claude 시나리오 설계 정상 Case

NORMAL

세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다.

- 01 — materials/day2/Codex_Claude_대화_시나리오.md 입력과 command가 기록됐는가
- 02 — 05_workflow_runs.json schema가 validate되는가
- 03 — Codex·Claude 시나리오 설계 focused test가 실제로 통과했는가

Codex·Claude 시나리오 설계 경계 Case

BOUNDARY

test 미실행·env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다.

- 01 — Codex·Claude 시나리오 설계: raw traceback 대신 stable error_code가 남는가
- 02 — 05_workflow_runs.json: 외부 쓰기·자동 메일이 false인가
- 03 — test 미실행·env 변경·외부 게시 추가 중 하나라도 있으면 HOLD다. 재현이 가능한가

Codex·Claude 시나리오 설계 현업 적용

ChatGPT/Codex 5.6 Sol ultra와 Claude 상위 모델을 개발 동료로 쓰는 작업법

- 01 — Codex·Claude 시나리오 설계은 합성·비식별 sample로 먼저 실행
- 02 — ChatGPT/Codex 5.6 Sol ultra와 Claude 상위 모델을 개발 동료로 쓰는 작업법의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 05_workflow_runs.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 05_workflow_runs.json을 운영 검토 자료로 사용

Codex·Claude 시나리오 설계 포트폴리오 적용

Codex·Claude 시나리오 설계 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — '알아서 Agent를 만들어줘'라고만 요청해 권한·비용·실패 경계가 사라진다.의 사용자 영향을 한 문장으로 설명
- 02 — Codex·Claude 시나리오 설계 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — Meet·ClovaNote·audio와 미승인 외부 쓰기 차단 test를 먼저 요청한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 05_workflow_runs.json과 README 재현 절차를 제출

Codex·Claude 시나리오 설계 운영 점검

품질

무엇을 세 시나리오가 같은 결과 계약과 다른 입력 경로를 사용한다.로 정의하는가

안전

Codex·Claude 시나리오 설계의 사람 승인과 external_write=false 위치

복구

'알아서 Agent를 만들어줘'라고만 요청해 권한·비용·실패 경계가 사라진다. 발생 시 05_workflow_runs.json에서 원인 상태를 확인

관측

05_workflow_runs.json에서 어떤 status·error_code를 보는가

05_workflow_runs.json 실행 증거

- 01 — 설명: Codex·Claude 시나리오 설계이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git diff -- src/course_services/day2_meeting_workflow.py tests/test_day2_meeting_workflow.py`
- 03 — 증거: 05_workflow_runs.json과 정상·실패 test 결과가 남아 있다.

다음 차시: LLM Provider와 비용 경계

DAY 2 · 6차시

15:00-15:50 (쉬는 시간·Q&A 17:30-18:00)

LLM Provider와 비용 경계

Provider · API opt-in · 06_provider_diagnostics.json

진행

LLM Provider와 비용 경계
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/openai_provider.py
src/course_services/
day2_meeting_workflow.py

확인할 결과

06_provider_diagnostics.json

6차시 입력과 결과

입력	————	05_workflow_runs.json
이번 차시	————	LLM Provider와 비용 경계
결과	————	06_provider_diagnostics.json
다음 연결	————	LangGraph 사람 검증

LLM Provider와 비용 경계 필요성

Agent는 여러 번 생각하고 도구를 호출해 단일 LLM 요약보다 비용·지연·오류 면적이 커진다.

provider 진단에서 06_provider_diagnostics.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 06_provider_diagnostics.json

LLM Provider와 비용 경계 학습 결과

- 01 — fixture·Ollama·OpenAI API·Codex CLI·Claude Code를 같은 계약 뒤에 두고 비용과 가용성을 표시한다.
- 02 — API key를 Notebook에 적거나 모델 미지원 오류를 fixture 성공으로 숨긴다. 왜 위험한지 설명한다.
- 03 — 06_provider_diagnostics.json에서 정상과 실패 상태를 직접 확인한다.

LLM Provider와 비용 경계 용어

용어	이 수업에서 쓰는 뜻
Provider	모델을 실행하는 서비스나 로컬 엔진
API opt-in	키·비용·전송을 알고 명시적으로 켜는 경로
Fallback	실패 시 정해진 대체 경로
Budget	호출 수·token·시간 한도

06_provider_diagnostics.json 입출력

INPUT

src/openai_provider.py

OUTPUT

06_provider_diagnostics.json

PROCESS

provider 진단 → model 가용성 → 비용 한도

VERIFY

선택 provider가 MeetingRecord schema와
provider_used를 반환한다.
key 미설정·model 미지원·timeout이 secret 노출 없이
stable reason으로 남는다.

LLM Provider와 비용 경계 실행 흐름

01

provider 진단

02

model 가용성

03

비용 한도

04

schema 검증

05

fallback reason

마지막 판단은 06_provider_diagnostics.json에서 사람이 확인합니다.

06_provider_diagnostics.json 정상·실패 계약

정상	——	선택 provider가 MeetingRecord schema와 provider_used를 반환한다.
경계	——	key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다.
외부 쓰기	——	LLM Provider와 비용 경계: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·06_provider_diagnostics.json

LLM Provider와 비용 경계 파일 지도

입력·fixture

src/openai_provider.py

구현 코드

src/course_services/day2_meeting_workflow.py

검증·test

.env.sample

따라하기 notebook

output/course-labs/day2-v2/06_provider_diagnostics.json

LLM Provider와 비용 경계 개념·코드

Provider

모델을 실행하는 서비스나 로컬 엔진

```
src/openai_provider.py
```

API opt-in

키·비용·전송을 알고 명시적으로 켜는 경로

```
src/course_services/day2_meeting_workflow.py
```

Fallback

실패 시 정해진 대체 경로

```
.env.sample
```

Budget

호출 수·token·시간 한도

```
output/course-labs/day2-v2/06_provider_diagnostics.json
```


LLM Provider와 비용 경계 선택 기준

구분	역할	이번 차시의 판단 기준
Provider	provider 진단 단계의 입력 기준	결정론 test로 먼저 확인
API opt-in	model 가용성 단계의 교체 경계	adapter 경계를 확인
Fallback	비용 한도 단계의 운영 조건	비용·지연·권한을 확인
Budget	schema 검증 단계의 판단 기록	사람 판단과 운영 기록을 확인

LLM Provider와 비용 경계 대표 실패

실패

API key를 Notebook에 적거나 모델 미지원 오류를 fixture 성공으로 숨긴다.

사용자 영향

API key를 Notebook에 적거나 모델 미지원 오류를 fixture 성공으로 숨긴다. 상태에서는 06_provider_diagnostics.json을 운영 판단에 사용할 수 없다.

LLM Provider와 비용 경계 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

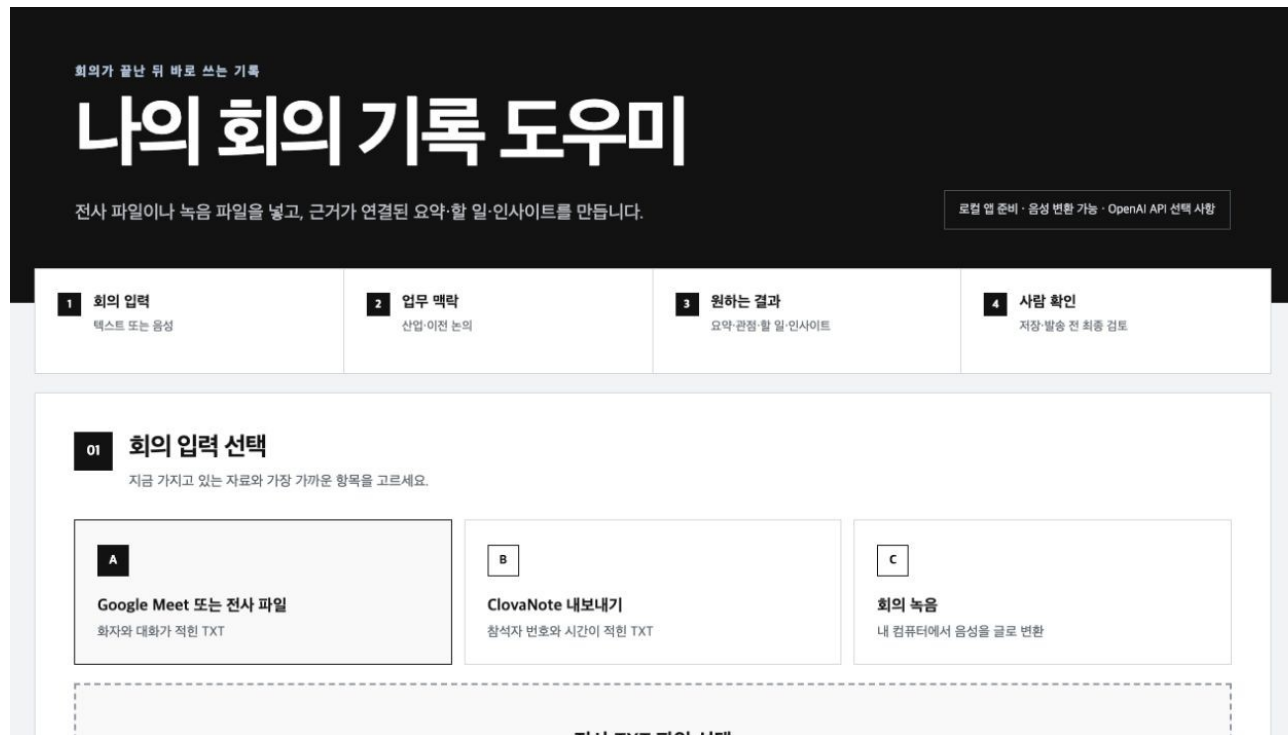
LLM Provider와 비용 경계 복구 경로

- 01 — 탐지: key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다.
- 02 — 중단: API key를 Notebook에 적거나 모델 미지원 오류를 fixture 성공으로 숨긴다.
- 03 — 복구: key는 env로만 읽고 requested·used provider·model·fallback_reason을 남긴다.
- 04 — 증거: 06_provider_diagnostics.json과 test 결과를 함께 남긴다.

LLM Provider와 비용 경계 Codex 검증 루프

src/course_services/
day2_meeting_workflow.py의
책임을 찾고
06_provider_diagnostics.json
계약을 focused test로 고정합니다.

LLM Provider와 비용 경계
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



LLM Provider와 비용 경계 Codex 작업 명세

목표

gpt-5.6-luna를 기본 후보로 두되 실제 계정 가용성과 auth·model·timeout 오류를 검증한다.

허용 경로

src/course_services/day2_meeting_workflow.py
.env.sample

완료 조건

선택 provider가 MeetingRecord schema와 provider_used를 반환한다.
key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다.

금지

LLM Provider와 비용 경계: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

LLM Provider와 비용 경계 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | LLM Provider와 비용 경계 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 선택 provider가 MeetingRecord schema와 provider_used를 반환한다. |
| 03 | Claude review | —— | 06_provider_diagnostics.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | LLM Provider와 비용 경계의 두 LLM 의견과 test는 자동 승인이 아니다 |

LLM Provider와 비용 경계 정상 시연

- 01 — 열기: `src/openai_provider.py`
- 02 — 구현 확인: `src/course_services/day2_meeting_workflow.py`
- 03 — 실행: `codex login status`
- 04 — 성공 신호: 선택 provider가 MeetingRecord schema와 provider_used를 반환한다.

LLM Provider와 비용 경계 실행 명령

```
$ codex login status
```

실행 전 확인

LLM Provider와 비용 경계: repository root · Python 3.12 · .env 미출력

06_provider_diagnostics.json 실행 결과

```
{
  "options": {
    "ollama": {
      "command": "ollama",
      "opt_in_example": "ollama run qwen3:4b",
      "role": "로컬 LLM 선택 실습",
      "installed": true,
      "status": "INSTALLED_NOT_EXECUTED",
      "auth_checked": false,
      "command_executed": false,
      "credential_value_read": false,
      "external_write": false
    },
    "codex": {
      "command": "codex",
      "opt_in_example": "codex exec --ephemeral --sandbox read-only '<요청>'",
      "role": "저장소 분석·코드 생성 Harness",
      "installed": true,
      "status": "INSTALLED_NOT_EXECUTED",
      "auth_checked": false,
      "command_executed": false,
      "credential_value_read": false,
      "external_write": false
    },
    "claude_code": {
      ...
    }
  }
}
```

LLM Provider와 비용 경계 실패·복구 시연

재현

key 미설정·model 미지원·timeout이
secret 노출 없이 stable reason으로
남는다.

복구

key는 env로만 읽고 requested·used
provider·model·fallback_reason을
남긴다.

확인: key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다.

LLM Provider와 비용 경계 소프트웨어 제작

열 파일	src/course_services/day2_meeting_workflow.py .env.sample
작업 범위	LLM Provider와 비용 경계 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	선택 provider가 MeetingRecord schema와 provider_used를 반환한다. 06_provider_diagnostics.json 저장

LLM Provider와 비용 경계 Notebook 셀

materials/day2/day2_service_lab.ipynb · 6차시

```
import os
from types import SimpleNamespace

provider_options = diagnose_provider_options()
cli_dry_runs = {
    name: run_optional_cli_prompt(name, "현재 회의 기록을 검토해 주세요.")
    for name in ("ollama", "codex", "claude_code")
}
RUN_OPENAI_LIVE = False
openai_result = run_optional_openai_record(
    envelopes["google_meet_text"], domain_context,
    env=os.environ if RUN_OPENAI_LIVE else {},
    model=os.getenv("OPENAI_MODEL", DEFAULT_OPENAI_MODEL),
    allow_fixture_fallback=True,
)

class LocalModelNotFound(Exception):
    status_code = 404

class FakeResponses:
    # ... 다음 줄은 Notebook에서 계속
```

LLM Provider와 비용 경계 Terminal 실행

```
$ codex login status
```

저장 위치

```
output/course-labs/day2-v2/06_provider_diagnostics.json
```

선택 provider가 MeetingRecord schema와 provider_used를 반환한다.
실패 시 06_provider_diagnostics.json의 status:error_code를 먼저 확인

LLM Provider와 비용 경계 실패 증거

key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다.

```
result_file = "output/course-labs/day2-v2/06_provider_diagnostics.json"
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"
external_write = false
```

06_provider_diagnostics.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

LLM Provider와 비용 경계 정상 Case

NORMAL

선택 provider가 MeetingRecord schema와 provider_used를 반환한다.

- 01 — src/openai_provider.py 입력과 command가 기록됐는가
- 02 — 06_provider_diagnostics.json schema가 validate되는가
- 03 — LLM Provider와 비용 경계 focused test가 실제로 통과했는가

LLM Provider와 비용 경계 경계 Case

BOUNDARY

key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다.

- 01 — LLM Provider와 비용 경계: raw traceback 대신 stable error_code가 남는가
- 02 — 06_provider_diagnostics.json: 외부 쓰기·자동 메일이 false인가
- 03 — key 미설정·model 미지원·timeout이 secret 노출 없이 stable reason으로 남는다. 재현이 가능한가

LLM Provider와 비용 경계 현업 적용

로컬 프라이버시·구독형 CLI·종량제 API를 상황별로 바꾸는 모델층

- 01 — LLM Provider와 비용 경계은 합성·비식별 sample로 먼저 실행
- 02 — 로컬 프라이버시·구독형 CLI·종량제 API를 상황별로 바꾸는 모델층의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 06_provider_diagnostics.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 06_provider_diagnostics.json을 운영 검토 자료로 사용

LLM Provider와 비용 경계 포트폴리오 적용

LLM Provider와 비용 경계 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — API key를 Notebook에 적거나 모델 미지원 오류를 fixture 성공으로 숨긴다.의 사용자 영향을 한 문장으로 설명
- 02 — LLM Provider와 비용 경계 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — gpt-5.6-luna를 기본 후보로 두되 실제 계정 가용성과 auth·model·timeout 오류를 검증한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 06_provider_diagnostics.json과 README 재현 절차를 제출

LLM Provider와 비용 경계 운영 점검

품질

무엇을 선택 provider가 MeetingRecord schema와 provider_used를 반환한다.로 정의하는가

안전

LLM Provider와 비용 경계의 사람 승인과 external_write=false 위치

복구

API key를 Notebook에 적거나 모델 미지원 오류를 fixture 성공으로 숨긴다. 발생 시 06_provider_diagnostics.json에서 원인 상태를 확인

관측

06_provider_diagnostics.json에서 어떤 status:error_code를 보는가

06_provider_diagnostics.json 실행 증거

- 01 — 설명: LLM Provider와 비용 경계이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: codex login status
- 03 — 증거: 06_provider_diagnostics.json과 정상·실패 test 결과가 남아 있다.

다음 차시: LangGraph 사람 검증

DAY 2 · 7차시

15:50-16:40 (쉬는 시간·Q&A 17:30-18:00)

LangGraph 사람 검증

State · Node · 07_human_review.json

진행

LangGraph 사람 검증
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/
day2_meeting_workflow.py
materials/day2/day2_service_lab.ipynb

확인할 결과

07_human_review.json

7차시 입력과 결과

입력 ————— 06_provider_diagnostics.json

이번 차시 ————— **LangGraph 사람 검증**

결과 ————— 07_human_review.json

다음 연결 ————— Desktop 선물 패키지

LangGraph 사람 검증 필요성

LangGraph는 AI가 아니라 AI와 코드가 어디서 멈추고 다시 시작할지 표현하는 상태 기계다.

입력 정규화에서 07_human_review.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 07_human_review.json

LangGraph 사람 검증 학습 결과

- 01 — 고정 순서와 예외 분기를 Graph로 보이고 approve·edit·reject 이후만 초안을 저장한다.
- 02 — READY를 사람 승인으로 오해해 Notion·Confluence·이메일에 바로 쓴다. 왜 위험한지 설명한다.
- 03 — 07_human_review.json에서 정상과 실패 상태를 직접 확인한다.

LangGraph 사람 검증 용어

용어	이 수업에서 쓰는 뜻
State	각 단계가 공유하는 현재 결과
Node	하나의 책임을 실행하는 단계
Interrupt	사람 판단을 기다리며 멈추는 지점
Draft	승인 전이라 외부에 반영되지 않은 결과

07_human_review.json 입출력

INPUT

src/course_services/day2_meeting_workflow.py

OUTPUT

07_human_review.json

PROCESS

입력 정규화 → 맥락 계획 → 회의록 생성

VERIFY

approve는 MD·email draft를 만들지만
external_write=false를 유지한다.
reject·unknown evidence·정책 위반은 외부 반영 없이
HOLD다.

LangGraph 사람 검증 실행 흐름

01

입력 정규화

02

맥락 계획

03

회의록 생성

04

evidence 검증

05

interrupt·초안

마지막 판단은 07_human_review.json에서 사람이 확인합니다.

07_human_review.json 정상·실패 계약

정상	——	approve는 MD·email draft를 만들지만 external_write=false를 유지한다.
경계	——	reject·unknown evidence·정책 위반은 외부 반영 없이 HOLD다.
외부 쓰기	——	LangGraph 사람 검증: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·07_human_review.json

LangGraph 사람 검증 파일 지도

입력·fixture

```
src/course_services/day2_meeting_workflow.py
```

구현 코드

```
materials/day2/day2_service_lab.ipynb
```

검증·test

```
tests/test_day2_meeting_workflow.py
```

따라하기 notebook

```
output/course-labs/day2-v2/07_human_review.json
```

LangGraph 사람 검증 개념·코드

State

각 단계가 공유하는 현재 결과

```
src/course_services/day2_meeting_workflow.py
```

Interrupt

사람 판단을 기다리며 멈추는 지점

```
tests/test_day2_meeting_workflow.py
```

Node

하나의 책임을 실행하는 단계

```
materials/day2/day2_service_lab.ipynb
```

Draft

승인 전이라 외부에 반영되지 않은 결과

```
output/course-labs/day2-v2/07_human_review.json
```

LangGraph 사람 검증 선택 기준

구분	역할	이번 차시의 판단 기준
State	입력 정규화 단계의 입력 기준	결정론 test로 먼저 확인
Node	맥락 계획 단계의 교체 경계	adapter 경계를 확인
Interrupt	회의록 생성 단계의 운영 조건	비용·지연·권한을 확인
Draft	evidence 검증 단계의 판단 기록	사람 판단과 운영 기록을 확인

LangGraph 사람 검증 대표 실패

실패

READY를 사람 승인으로 오해해
Notion·Confluence·이메일에 바로
쓴다.

사용자 영향

READY를 사람 승인으로 오해해
Notion·Confluence·이메일에 바로 쓴다.
상태에서는 07_human_review.json을 운영
판단에 사용할 수 없다.

LangGraph 사람 검증 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

LangGraph 사람 검증 복구 경로

- 01 — 탐지: reject·unknown evidence·정책 위반은 외부 반영 없이 HOLD다.
- 02 — 중단: READY를 사람 승인으로 오해해 Notion·Confluence·이메일에 바로 쓴다.
- 03 — 복구: READY_FOR_REVIEW와 APPROVED를 다른 상태로 두고 외부 adapter를 호출하지 않는다.
- 04 — 증거: 07_human_review.json과 test 결과를 함께 남긴다.

LangGraph 사람 검증 Codex 검증 루프

materials/day2/
day2_service_lab.ipynb의 책임을
찾고 07_human_review.json
계약을 focused test로
고정합니다.

LangGraph 사람 검증
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

WORKFLOW LAB

Pipeline Review Result

02 LANGGRAPH INTERRUPT

결과를 내보내기 전에 사람이 결정합니다.

REVIEW_REQUIRED

고객 문의 자동화 PoC 범위 회의

배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.

ACTION ITEMS

공개 FAQ 30건 비식별 샘플 정리 서연 · 2026-08-27
근거 s12, s18

응답 JSON Schema와 실패 테스트 작성 준호 · 2026-08-28
근거 s27, s31

interrupt_payload.json read only

```
{
  "question": "회의 결과를 승인·수정·거절하시겠습니까?",
  "request_id": "meeting-approve-001",
  "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외  
부 발행 전 사람이 검토한다.",
  "action_items": [
    {
      "task": "공개 FAQ 30건 비식별 샘플 정리",
      "owner": "서연",
      "due_date": "2026-08-27",
      "evidence_ids": [
        "s12",
        "s18"
      ]
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호"
```

LangGraph 사람 검증 Codex 작업 명세

목표

approve·edit·reject·unknown evidence·미승인 export를 각각 상태 전이 test로 고정한다.

허용 경로

materials/day2/day2_service_lab.ipynb
tests/test_day2_meeting_workflow.py

완료 조건

approve는 MD·email draft를 만들지만 external_write=false를 유지한다.
reject·unknown evidence·정책 위반은 외부 반영 없이 HOLD다.

금지

LangGraph 사람 검증: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

LangGraph 사람 검증 독립 리뷰

- 01 **Codex diff** — LangGraph 사람 검증 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — approve는 MD·email draft를 만들지만 external_write=false를 유지한다.
- 03 **Claude review** — 07_human_review.json 변경 라인·누락 경계를 별도 session에서 검토
- 04 **Human merge** — LangGraph 사람 검증의 두 LLM 의견과 test는 자동 승인이 아니다

LangGraph 사람 검증 정상 시연

- 01 — 열기: `src/course_services/day2_meeting_workflow.py`
- 02 — 구현 확인: `materials/day2/day2_service_lab.ipynb`
- 03 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py -k review`
- 04 — 성공 신호: approve는 MD·email draft를 만들지만 `external_write=false`를 유지한다.

LangGraph 사람 검증 실행 명령

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py -k review
```

실행 전 확인

LangGraph 사람 검증: repository root · Python 3.12 · .env 미출력

07_human_review.json 실행 결과

```
{
  "decisions": {
    "approve": {
      "status": "DRAFT_READY",
      "review": {
        "decision": "approve",
        "status": "APPROVED_READY_FOR_DRAFT",
        "export_ready": true,
        "human_reviewed": true,
        "external_write": false
      },
      "export_status": "DRAFT_READY",
      "external_write": false
    },
    "edit": {
      "status": "DRAFT_READY",
      "review": {
        "decision": "edit",
        "status": "EDITED_READY_FOR_DRAFT",
        "export_ready": true,
        "human_reviewed": true,
        "external_write": false
      },
      "export_status": "DRAFT_READY",
      "external_write": false
    },
    ...
  }
}
```

LangGraph 사람 검증 실패·복구 시연

재현

reject-unknown evidence-정책 위반은
외부 반영 없이 HOLD다.

복구

READY_FOR_REVIEW와 APPROVED
를 다른 상태로 두고 외부 adapter를
호출하지 않는다.

확인: reject-unknown evidence-정책 위반은 외부 반영 없이 HOLD다.

LangGraph 사람 검증 소프트웨어 제작

열 파일	<code>materials/day2/day2_service_lab.ipynb</code> <code>tests/test_day2_meeting_workflow.py</code>
작업 범위	LangGraph 사람 검증 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	approve는 MD·email draft를 만들지만 <code>external_write=false</code> 를 유지한다. <code>07_human_review.json</code> 저장

LangGraph 사람 검증 Notebook 셀

materials/day2/day2_service_lab.ipynb · 7차시

```
review_runs = {
    "approve": run_meeting_workflow(
        sources["google_meet_text"], domain_context, review_decision="approve"
    ),
    "edit": run_meeting_workflow(
        sources["google_meet_text"], domain_context, review_decision="edit",
        review_edits={
            "meeting_summary": "사람이 근거를 확인하고 배송 지연 기록 범위를 수정했습니다.",
            "todo_updates": {"0": {"owner": "민지", "due_date": "2026-09-05"}},
        },
    ),
    "reject": run_meeting_workflow(
        sources["google_meet_text"], domain_context, review_decision="reject"
    ),
}
boundary_record = MeetingRecord.model_validate(review_runs["approve"]["record"])
boundary_payload = boundary_record.model_dump(mode="json")
boundary_payload["todos"][0]["evidence_ids"] = ["s999"]
evidence_boundary = validate_record_evidence(
    MeetingRecord.model_validate(boundary_payload),
# ... 다음 줄은 Notebook에서 계속
```

LangGraph 사람 검증 Terminal 실행

```
$ python -m pytest -q tests/test_day2_meeting_workflow.py -k review
```

저장 위치

```
output/course-labs/day2-v2/07_human_review.json
```

approve는 MD-email draft를 만들지만 external_write=false를 유지한다.
실패 시 07_human_review.json의 status:error_code를 먼저 확인

LangGraph 사람 검증 실패 증거

reject-unknown evidence-정책 위반은 외부 반영 없이 HOLD다.

```
result_file = "output/course-labs/day2-v2/07_human_review.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

07_human_review.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

LangGraph 사람 검증 정상 Case

NORMAL

approve는 MD·email draft를 만들지만 external_write=false를 유지한다.

- 01 — src/course_services/day2_meeting_workflow.py 입력과 command가 기록됐는가
- 02 — 07_human_review.json schema가 validate되는가
- 03 — LangGraph 사람 검증 focused test가 실제로 통과했는가

LangGraph 사람 검증 경계 Case

BOUNDARY

reject·unknown evidence·정책 위반은 외부 반영 없이 HOLD다.

- 01 — LangGraph 사람 검증: raw traceback 대신 stable error_code가 남는가
- 02 — 07_human_review.json: 외부 쓰기·자동 메일이 false인가
- 03 — reject·unknown evidence·정책 위반은 외부 반영 없이 HOLD다. 재현이 가능한가

LangGraph 사람 검증 현업 적용

원문·담당·기한·톤을 확인한 뒤에만 공유를 준비하는 흐름

- 01 — LangGraph 사람 검증은 합성·비식별 sample로 먼저 실행
- 02 — 원문·담당·기한·톤을 확인한 뒤에만 공유를 준비하는 흐름의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 07_human_review.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 07_human_review.json을 운영 검토 자료로 사용

LangGraph 사람 검증 포트폴리오 적용

LangGraph 사람 검증 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — READY를 사람 승인으로 오해해 Notion·Confluence·이메일에 바로 쓴다.의 사용자 영향을 한 문장으로 설명
- 02 — LangGraph 사람 검증 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — approve·edit·reject·unknown evidence·미승인 export를 각각 상태 전이 test로 고정한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 07_human_review.json과 README 재현 절차를 제출

LangGraph 사람 검증 운영 점검

품질

무엇을 approve는 MD·email draft를 만들지만 external_write=false를 유지한다.로 정의하는가

복구

READY를 사람 승인으로 오해해 Notion·Confluence·이메일에 바로 쓴다. 발생 시 07_human_review.json에서 원인 상태를 확인

안전

LangGraph 사람 검증의 사람 승인과 external_write=false 위치

관측

07_human_review.json에서 어떤 status·error_code를 보는가

07_human_review.json 실행 증거

- 01 — 설명: LangGraph 사람 검증이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_day2_meeting_workflow.py -k review`
- 03 — 증거: 07_human_review.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Desktop 선물 패키지

DAY 2 · 8차시

16:40-17:30 (쉬는 시간·Q&A 17:30-18:00)

Desktop 선물 패키지

Desktop app · Container · 08_export_drafts.json

진행

Desktop 선물 패키지
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

desktop-app/meeting-intelligence/
README.md
desktop-app/meeting-intelligence/docker-
compose.yml

확인할 결과

08_export_drafts.json

8차시 입력과 결과

입력	————	07_human_review.json
이번 차시	————	Desktop 선물 패키지
결과	————	08_export_drafts.json
다음 연결	————	17:30-18:00 쉬는 시간·Q&A

Desktop 선물 패키지 필요성

일반인에게는 Notebook 성공보다 설치·첫 실행·오류 복구·결과 저장이
서비스의 품질이다.

입력 선택에서 08_export_drafts.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: 08_export_drafts.json

Desktop 선물 패키지 학습 결과

- 01 — 세 입력·도메인 맥락·결과 선택·사람 검증을 macOS·Windows 앱으로 전달한다.
- 02 — 강사의 로그인·캐시·경로에서만 되는 Demo를 배포 가능한 제품으로 오해한다. 왜 위험한지 설명한다.
- 03 — 08_export_drafts.json에서 정상과 실패 상태를 직접 확인한다.

Desktop 선물 패키지 용어

용어	이 수업에서 쓰는 뜻
Desktop app	일반인이 입력과 결과를 다루는 로컬 프로그램
Container	같은 실행 환경을 묶은 단위
Integration plan	승인 후 어떤 외부 도구에 쓸지의 계획
Unsigned	조직 코드 서명이 아직 없는 교육용 패키지

08_export_drafts.json 입출력

INPUT

desktop-app/meeting-intelligence/README.md

OUTPUT

08_export_drafts.json

PROCESS

입력 선택 → 도메인 맥락 → Workflow-Agent

VERIFY

세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.
미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.

Desktop 선물 패키지 실행 흐름

01

입력 선택

02

도메인 맥락

03

Workflow-Agent

04

사람 검증

05

MD-email 초안

마지막 판단은 08_export_drafts.json에서 사람이 확인합니다.

08_export_drafts.json 정상·실패 계약

정상	—— 세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.
경계	—— 미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.
외부 쓰기	—— Desktop 선물 패키지: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·08_export_drafts.json

Desktop 선물 패키지 파일 지도

입력·fixture

desktop-app/meeting-intelligence/README.md

구현 코드

desktop-app/meeting-intelligence/docker-compose.yml

검증·test

materials/day2/day2_service_lab.ipynb

따라하기 notebook

output/course-labs/day2-v2/08_export_drafts.json

Desktop 선물 패키지 개념·코드

Desktop app

일반인이 입력과 결과를 다루는 로컬 프로그램

```
desktop-app/meeting-intelligence/README.md
```

Integration plan

승인 후 어떤 외부 도구에 쓸지의 계획

```
materials/day2/day2_service_lab.ipynb
```

Container

같은 실행 환경을 묶은 단위

```
desktop-app/meeting-intelligence/docker-compose.yml
```

Unsigned

조직 코드 서명이 아직 없는 교육용 패키지

```
output/course-labs/day2-v2/08_export_drafts.json
```

Desktop 선물 패키지 선택 기준

구분	역할	이번 차시의 판단 기준
Desktop app	입력 선택 단계의 입력 기준	결정론 test로 먼저 확인
Container	도메인 맥락 단계의 교체 경계	adapter 경계를 확인
Integration plan	Workflow·Agent 단계의 운영 조건	비용·지연·권한을 확인
Unsigned	사람 검증 단계의 판단 기록	사람 판단과 운영 기록을 확인

Desktop 선물 패키지 대표 실패

실패

강사의 로그인·캐시·경로에서만 되는 Demo를 배포 가능한 제품으로 오해한다.

사용자 영향

강사의 로그인·캐시·경로에서만 되는 Demo를 배포 가능한 제품으로 오해한다. 상태에서는 08_export_drafts.json을 운영 판단에 사용할 수 없다.

Desktop 선물 패키지 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

Desktop 선물 패키지 복구 경로

- 01 — 탐지: 미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.
- 02 — 중단: 강사의 로그인·캐시·경로에서만 되는 Demo를 배포 가능한 제품으로 오해한다.
- 03 — 복구: fixture 완주를 기본으로 두고 STT·Ollama·CLI·API를 선택 확장으로 분리한다.
- 04 — 증거: 08_export_drafts.json과 test 결과를 함께 남긴다.

Desktop 선물 패키지 Codex 검증 루프

desktop-app/meeting-intelligence/docker-compose.yml의 책임을 찾고
08_export_drafts.json 계약을
focused test로 고정합니다.

Desktop 선물 패키지
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

06 검토할 결과

원문과 담당자-기한을 확인한 뒤에만 저장하거나 보내세요.

검토 준비

회의 기록 초안이 준비되었습니다

Google Meet 또는 전자 TXT · 인터넷 없이 연습용 결과

문서 저장

전체 결과 저장

아직 저장하거나 보내지 않았습니다. 원문, 담당자, 기한을 사람이 확인해야 합니다.

상황 판단 · Agent

이번 요청에 사용한 방식

외부 정보·도구·후속 전달 가능성이 있는 추가 요청이라 계획과 승인을 먼저 분리했습니다.

요청 목적 판단

→ 필요한 정보와 연결 후보 계획

→ 허용된 회의 입력만 사용

→ 선택 결과 생성

→ 외부 작업 계획 분리

→ 근거 번호 검사

→ 사람 검토 대기

8 원문 구간	3 확인할 참석자	5 할 일	7 연결된 근거
------------	--------------	----------	-------------

회의 기록

문서 미리보기

이메일 초안

연결 계획

원문 근거

회의 기록과 후속 실행

회의에서는 다음 내용을 중심으로 논의했습니다. 오늘은 고객 상담 요약 기능의 1차 출시 범위와 검증 기준을 정하겠습니다. 상담 후 요약, 고객 요청, 상담사가 할 일을 한 화면에 보여주는 범위가 적절합니다. 개인정보가 들어갈 수 있으니 원문 보관 기간과 마스킹 기준을 먼저 확인해야 합니다. 1차 출시는 자동 발송 없이 상담사가 결과를 확인하고 저장하는 방

Desktop 선물 패키지 Codex 작업 명세

목표

API·UI·Docker·Windows EXE·macOS PKG가 같은 schema와 port를 쓰는지 검증한다.

허용 경로

desktop-app/meeting-intelligence/docker-compose.yml
materials/day2/day2_service_lab.ipynb

완료 조건

세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.
미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.

금지

Desktop 선물 패키지: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Desktop 선물 패키지 독립 리뷰

- | | | | |
|----|---------------|----|---|
| 01 | Codex diff | —— | Desktop 선물 패키지 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다. |
| 03 | Claude review | —— | 08_export_drafts.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Desktop 선물 패키지의 두 LLM 의견과 test는 자동 승인이 아니다 |

Desktop 선물 패키지 정상 시연

- 01 — 열기: `desktop-app/meeting-intelligence/README.md`
- 02 — 구현 확인: `desktop-app/meeting-intelligence/docker-compose.yml`
- 03 — 실행: `cd desktop-app/meeting-intelligence && python -m pytest -q && go test ./...`
- 04 — 성공 신호: 세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.

Desktop 선물 패키지 실행 명령

```
$ cd desktop-app/meeting-intelligence && python -m pytest -q && go test ./...
```

실행 전 확인

Desktop 선물 패키지: repository root · Python 3.12 · .env 미출력

08_export_drafts.json 실행 결과

```
{
  "markdown_files": {
    "google_meet_text": "output/course-labs/day2-v2/08_google_meet_text_meeting.md",
    "clovanote_txt": "output/course-labs/day2-v2/08_clovanote_txt_meeting.md",
    "audio_stt": "output/course-labs/day2-v2/08_audio_stt_meeting.md"
  },
  "email_drafts": {
    "google_meet_text": {
      "subject": "[회의 기록] 배송 지연 회의 기록 자동화 범위 확정 회의 기록",
      "audience": "internal",
      "to": [],
      "body": "안녕하세요, 회의 참석자 여러분.\n\n배송 지연 회의 기록 자동화 범위 확정 관련 회의 내용을 아래와 같이 정리했습니다.\n\n오늘은 배송 지연 회의 기록 자동화 범위를 확정하겠습니다. WISMO 문의를 우선 처리하고 환불 자동화는 보류하는 것이 좋겠습니다.\n\n[후속 조치]\n- 제가 9월 2일까지 고객 안내 문구를 정리해 공유하겠습니다. / 서연 / 2026-09-02\n\n담당자·기한·대외 공유 범위를 확인한 뒤 발송해 주세요.",
      "send": false,
      "external_write": false,
      "human_recipient_check_required": true
    },
    "clovanote_txt": {
      "subject": "[회의 기록] 배송 지연 회의 기록 자동화 범위 확정 회의 기록",
      "audience": "internal",
      "to": [],
      "body": "안녕하세요, 회의 참석자 여러분.\n\n배송 지연 회의 기록 자동화 범위 확정 관련 회의 내용을 아래와 같이 정리했습니다.\n\n배송 지연 원인 분류를 1차 범위로 확정합니다. 운영팀 부담을 확인한 뒤 다음 범위를 결정하겠습니다.\n\n[후속 조치]\n- 제가 9월 3일까지 분류 기준을 작성하겠습니다. / 준호 / 2026-09-03\n\n담당자·기한·대외 공유 범위를 확인한 뒤 발송해 주세요.",
      "send": false,
      "external_write": false,
      "human_recipient_check_required": true
    },
    ...
  }
}
```

Desktop 선물 패키지 실패·복구 시연

재현

미로그인·STT 실패·미승인·MCP 쓰기
요청은 외부 쓰기 없이 HOLD다.

복구

fixture 완주를 기본으로 두고
STT·Ollama·CLI·API를 선택 확장으로
분리한다.

확인: 미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.

Desktop 선물 패키지 소프트웨어 제작

열 파일

desktop-app/meeting-intelligence/docker-compose.yml
materials/day2/day2_service_lab.ipynb

작업 범위

Desktop 선물 패키지 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.
08_export_drafts.json 저장

Desktop 선물 패키지 Notebook 셀

materials/day2/day2_service_lab.ipynb · 8차시

```
markdown_files = {}
email_drafts = {}
for name, result in workflow_runs.items():
    record = MeetingRecord.model_validate(result["record"])
    markdown_text = result["exports"]["markdown"]
    markdown_path = save_text(f"08_{name}_meeting.md", markdown_text)
    markdown_files[name] = str(markdown_path.relative_to(ROOT))
    email_drafts[name] = render_email_draft(record, audience="internal")

save_json("08_email_drafts.json", email_drafts)
focused_test = run_command(
    sys.executable, "-m", "pytest", "-q", "tests/test_day2_meeting_workflow.py"
)
day1_suite = run_command(
    sys.executable, "-m", "pytest", "-q",
    "tests/test_day1_agent.py",
    "tests/test_langchain_langgraph_lab.py",
    "tests/test_meeting_agent_workflow.py",
    "tests/test_openai_provider.py",
    "tests/test_ollama_tool_agent.py",
    # ... 다음 줄은 Notebook에서 계속
```


Desktop 선물 패키지 Terminal 실행

```
$ cd desktop-app/meeting-intelligence && python -m pytest -q && go test ./...
```

저장 위치

output/course-labs/day2-v2/08_export_drafts.json

세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.
실패 시 08_export_drafts.json의 status·error_code를 먼저 확인

Desktop 선물 패키지 실패 증거

미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.

```
result_file = "output/course-labs/day2-v2/08_export_drafts.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

08_export_drafts.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Desktop 선물 패키지 정상 Case

NORMAL

세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.

- 01 — desktop-app/meeting-intelligence/README.md 입력과 command가 기록됐는가
- 02 — 08_export_drafts.json schema가 validate되는가
- 03 — Desktop 선물 패키지 focused test가 실제로 통과했는가

Desktop 선물 패키지 경계 Case

BOUNDARY

미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다.

- 01 — Desktop 선물 패키지: raw traceback 대신 stable error_code가 남는가
- 02 — 08_export_drafts.json: 외부 쓰기·자동 메일이 false인가
- 03 — 미로그인·STT 실패·미승인·MCP 쓰기 요청은 외부 쓰기 없이 HOLD다. 재현이 가능한가

Desktop 선물 패키지 현업 적용

도구가 달라도 자신의 맥락과 톤을 유지하는 개인용 회의기록 Agent

- 01 — Desktop 선물 패키지는 합성·비식별 sample로 먼저 실행
- 02 — 도구가 달라도 자신의 맥락과 톤을 유지하는 개인용 회의기록 Agent의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 08_export_drafts.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — 08_export_drafts.json을 운영 검토 자료로 사용

Desktop 선물 패키지 포트폴리오 적용

Desktop 선물 패키지 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 강사의 로그인·캐시·경로에서만 되는 Demo를 배포 가능한 제품으로 오해한다.의 사용자 영향을 한 문장으로 설명
- 02 — Desktop 선물 패키지 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — API·UI·Docker·Windows EXE·macOS PKG가 같은 schema와 port를 쓰는지 검증한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — 08_export_drafts.json과 README 재현 절차를 제출

Desktop 선물 패키지 운영 점검

품질

무엇을 세 입력 중 하나로 MeetingRecord·MD·email draft를 만들고 검토 상태를 보여준다.로 정의하는가

복구

강사의 로그인·캐시·경로에서만 되는 Demo를 배포 가능한 제품으로 오해한다. 발생 시 08_export_drafts.json에서 원인 상태를 확인

안전

Desktop 선물 패키지의 사람 승인과 external_write=false 위치

관측

08_export_drafts.json에서 어떤 status·error_code를 보는가

08_export_drafts.json 실행 증거

- 01 — 설명: Desktop 선물 패키지가 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `cd desktop-app/meeting-intelligence && python -m pytest -q && go test ./...`
- 03 — 증거: 08_export_drafts.json과 정상·실패 test 결과가 남아 있다.

17:30-18:00 쉬는 시간·Q&A